



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA - DISI

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

.....

.....

APPLICAZIONI WEB SERVERLESS SULL'INFRASTRUTTURA DI AMAZON WEB SERVICES

Tesi di laurea

Relatore

Prof. Paolo Bellavista

Presentata da

Christian Candela

Sessione dicembre 2023

Anno Accademico 2022/2023

Introduzione

Negli ultimi anni sono sempre di più le aziende che decidono di utilizzare il cloud per offrire i propri servizi informatici. Si evita quindi di possedere fisicamente tutti gli elementi hardware e di dover impostare le macchine virtuali e i container su cui far girare i software. Tutto viene spostato sulla “nuvola”, un’infrastruttura composta da vari data center, condivisa da vari clienti e messa a disposizione da un provider. Il leader del settore cloud in questi anni è Amazon Web Services che con le sue centinaia di servizi risponde a quasi tutte le necessità. Con l’infrastruttura di Amazon è possibile realizzare delle applicazioni definite serverless. Questo tipo di app è comunque eseguita su dei server ma non richiede che lo sviluppatore si occupi della loro gestione. Sarà compito del provider provvedere alla scalabilità delle risorse necessarie all’esecuzione, garantire la tolleranza ai guasti e l’affidabilità. Con la tecnologia serverless lo sviluppatore può concentrarsi unicamente sulla logica applicativa velocizzando di molto la realizzazione e la messa online del proprio prodotto. Ovviamente l’utilizzo di questa tecnica presenta sia aspetti negativi che positivi. È quindi necessario eseguire un’attenta analisi di ciò che si deve effettivamente realizzare e quali siano i requisiti che bisogna necessariamente rispettare.

Lo scopo di questo elaborato di tesi è quello di mostrare i servizi che Amazon Web Services mette a disposizione per la realizzazione di applicazioni serverless. Sarà presentata prima una descrizione generale di cosa sia il cloud computing e come questo si contrappone ad una organizzazione on-premise. Dopo questa analisi ci sarà una breve panoramica di AWS seguita da un approfondito studio sui vari servizi messi a disposizione dall’infrastruttura. In questo caso ci si focalizzerà soprattutto su quelli necessari per la realizzazione di app serverless. Grazie all’ampia documentazione fornita da Amazon si descriverà il loro funzionamento e le varie impostazioni presenti per personalizzare al massimo la loro esecuzione. Il tutto sarà accompagnato da esempi di codice. Verrà dato anche ampio spazio all’importanza della cooperazione dei vari elementi mostrati. Per costruire infatti un progetto più complesso non è possibile utilizzare singolarmente uno dei prodotti proposti. Per mostrare al meglio le funzionalità dell’ambiente cloud verrà presentata anche l’implementazione di una app.

Quest'ultima sfrutterà tutti i servizi esposti nei capitoli precedenti e sarà sottoposta a dei test per analizzarne l'efficienza e i tempi di esecuzione.

Il lavoro di tesi si articola quindi in tre diversi capitoli: nel primo è presenta un'analisi degli aspetti negativi e positivi delle scelte che un'azienda può adottare per esporre i propri servizi online (on-premise vs cloud). Nel secondo capitolo è invece presente un'introduzione sull'infrastruttura di Amazon Web Services. Questa è seguita dalla spiegazione dei prodotti scelti che può essere vista come un manuale di utilizzo di quest'ultimi. Infine, nel terzo capitolo si discute la realizzazione di un'applicazione che sfrutta i servizi descritti precedentemente e che illustra come implementare in ambiente cloud un'idea applicativa. Sempre nel terzo capitolo si svolgono delle analisi sui tempi di esecuzione del software implementato.

Indice

Introduzione	1
Capitolo 1.....	5
1.1 On-premise.....	5
1.2 Cloud computing.....	6
1.3 Obiettivi	7
1.4 Motivazioni	8
Capitolo 2.....	9
2.1 Amazon AWS	9
2.2 Applicazioni serverless	9
2.3 Servizi AWS	10
2.3.1 AWS Lambda	11
2.3.2 AWS IAM.....	16
2.3.3 Amazon API Gateway	20
2.3.4 AWS Amplify	24
2.3.5 Amazon Cognito	27
2.3.6 Amazon DynamoDB.....	29
2.3.7 AWS X-Ray.....	33
Capitolo 3.....	34
3.1 Applicazione	34
3.2 Abstract.....	34
3.3 Implementazione.....	35
3.3.1 Registrazione	35
3.3.2 Login.....	37
3.3.3 Area personale	39
3.3.4 Critta messaggi	41
3.3.5 Decritta messaggi.....	44
3.3.6 Home.....	47
3.3.7 Caricamento delle pagine.....	49
3.4 Analisi delle performance	49
Conclusioni	53
Indice delle figure	55
Indice delle tabelle	57

Bibliografia	57
--------------------	----

Capitolo 1

1.1 On-premise

In passato una delle soluzioni più adottate dalle aziende per offrire servizi informatici era di possedere fisicamente dei server. Questi venivano ospitati nella propria sede, da questo il termine “On-premise”, cioè, “nelle sedi”, e per accedervi bastava collegarsi alla rete interna aziendale. Per possedere server propri era dunque necessario acquistare l’hardware in anticipo (correttamente dimensionato in base al carico del servizio) e questo comportava un grande investimento in denaro. Ci si doveva occupare anche dell’installazione, della manutenzione, degli aggiornamenti, della tolleranza ai fallimenti, dell’affidabilità e della gestione di tutto ciò che ruotava intorno ai servizi. Diventavano responsabilità diretta della società anche la sicurezza dei dati e la scalabilità delle risorse in base al flusso di utenti. In caso di problemi di natura hardware che comportava la sostituzione di qualche componente era l’azienda stessa a dover investire nuovamente del denaro per la sostituzione. [1] Questa era e rimane una soluzione non adatta a piccole realtà che non possono permettersi un investimento iniziale troppo elevato. Sono ovviamente presenti anche aspetti positivi ed è per questo che è una soluzione ancora utilizzata. Si ha il completo controllo di tutta l’infrastruttura e anche dei dati salvati su di essa, utile a causa delle particolari legislazioni sulla condivisione degli stessi. In più non è necessario avere a disposizione una connessione ad Internet per comunicare con i server ed è possibile collegarsi alle macchine fisicamente visto che si trovano magari sotto qualche scrivania o in uno scantinato facilmente accessibile.

1.2 Cloud computing

Grazie al cloud computing è possibile utilizzare servizi informatici senza possedere l'infrastruttura. Quest'ultima è infatti offerta da società che hanno investito per costruire dei data center e che permettono, tramite una tariffazione a consumo, di acquisire spazio di archiviazione, potenza di calcolo e molto altro ancora. I vantaggi rispetto ad un approccio on-premise sono quindi un minor investimento iniziale. Ciò permette anche a piccole realtà di accedere alle tecnologie più recenti per realizzare e pubblicare la propria idea in breve tempo. Un altro aspetto da tenere in considerazione è che grazie al cloud si accede a servizi di scalabilità automatica. Questo comporta che nel caso in cui l'applicazione da un giorno all'altro passi da poche decine di utenti a migliaia non smetterà di funzionare ma semplicemente acquisirà maggiori risorse. Le società che offrono servizi cloud hanno spesso data center sparsi in tutto il mondo e quindi è possibile distribuire i propri servizi a livello globale. In questo modo, essendo più vicini agli utilizzatori, si ottiene una latenza minore che migliora di conseguenza l'usabilità. [2]

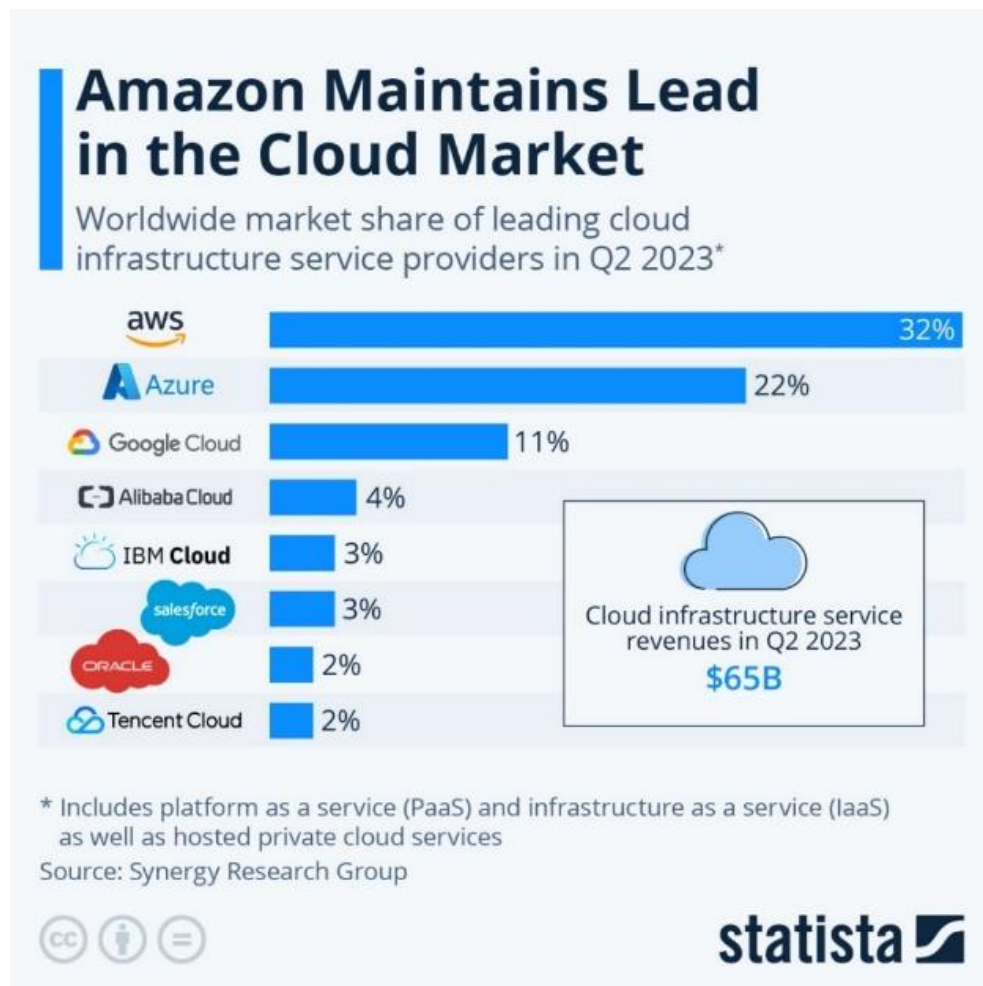
L'utilizzo di questa soluzione non è però priva di problematiche. Per usufruire dei servizi cloud è necessario possedere una connessione ad Internet in qualsiasi momento. In più si perde il controllo sui propri dati e si può incorrere in un lock-in, cioè una situazione in cui si desidera cambiare fornitore del servizio ma la migrazione è complessa a causa di mancanza di standard condivisi, ogni provider di solito ha una tecnologia proprietaria che non può essere trasportata sulle architetture di altri cloud.

Il cloud è una tecnologia che sempre più aziende stanno adottando, il mercato è quindi in crescita costante. Il suo valore è passato dai 293 miliardi di dollari nel 2019 ai 331 miliardi nel 2020. [3] Il leader dell'infrastruttura cloud è ormai da molti anni Amazon Web Services ¹(AWS) con una quota del 32%, seguita poi da Azure² al 22% e Google Cloud³ all'11%.

¹ <https://aws.amazon.com/it/>

² <https://azure.microsoft.com/en-us>

³ <https://cloud.google.com/>



4

Figura 1 - Mercato Cloud

1.3 Obiettivi

L'obiettivo di questo lavoro di tesi è di presentare il framework offerto da Amazon Web Services. Visto il vasto numero di servizi l'attenzione sarà posta soprattutto su quelli che permettono la realizzazione di applicazioni web serverless come ad esempio AWS Lambda. Si andrà quindi a descrivere in maniera dettagliata il loro funzionamento inserendo anche degli esempi di programmazione. Un altro aspetto centrale nel documento sarà mostrare come i vari servizi di AWS siano connessi e come collaborino per raggiungere un preciso scopo. Infine, sarà presentata

⁴<https://www.statista.com/chart/18819/worldwide-market-share-of-leading-cloud-infrastructure-service-providers/>

un'applicazione web costruita utilizzando i servizi precedentemente descritti. L'obiettivo è quello di mostrare gli aspetti positivi e negativi dell'utilizzo di una configurazione serverless anche tramite delle misurazioni delle performance.

1.4 Motivazioni

Ho deciso di sviluppare un progetto di tesi sull'infrastruttura cloud dato che sembra essere una delle tendenze più forti negli ultimi anni. Tra le varie società che offrono queste infrastrutture nella nuvola ho deciso di scegliere Amazon Web Services anche considerata la sua posizione attuale di leader del mercato. Inoltre, la sua vasta gamma di servizi mi ha sempre suscitato una certa curiosità. Credo anche che le conoscenze acquisite durante la stesura di questo documento potranno sia essere utili in ambito lavorativo sia essere un punto di partenza per un futuro approfondimento.

Capitolo 2

2.1 Amazon AWS

AWS è il servizio cloud offerto da Amazon che mette a disposizione più di 200 prodotti che rispondono alla maggior parte delle necessità degli utilizzatori. La società dispone di una rete globale composta da 32 regioni e ognuna di queste possiede diverse zone di disponibilità. In totale ne troviamo 102. Questo permette di distribuire facilmente il carico a livello globale. I servizi offerti spaziano dalla potenza di calcolo al machine learning passando per l'intelligenza artificiale e l'Internet of Things.

Un aspetto che rende molto concorrenziale i servizi offerti da Amazon è la velocità con il quale vengono sviluppate ed implementate le nuove tecnologie. Ad esempio, la società fu la prima a permettere lo sviluppo di applicazioni serverless sulla propria infrastruttura. Il lancio di questa nuova funzionalità avvenne nel 2014 sotto il nome di AWS Lambda. [4]

2.2 Applicazioni serverless

Il termine serverless sembrerebbe indicare che queste applicazioni funzionino senza l'utilizzo di un server. Tuttavia, questa interpretazione è errata. Il termine sta infatti ad indicare che chi si occupa dello sviluppo dell'app non deve preoccuparsi dell'infrastruttura che permette la sua esecuzione. Quest'ultima è gestita unicamente dal provider che la mette a disposizione. Lo sviluppatore deve interessarsi unicamente di scrivere ed implementare il codice del back-end senza interessarsi delle risorse che andrà ad utilizzare. Non ci sarà quindi bisogno di impiegare del tempo per capire di quanta potenza di calcolo o larghezza di banda si necessita. L'applicazione serverless scalerà in maniera autonoma acquisendo la quantità di risorse necessaria in base al numero di utenti che la sta utilizzando. Questo si traduce anche in una modifica dei costi dato che si paga solo per quello che si utilizza. Non è necessario quindi acquistare una specifica potenza di calcolo o un quantitativo di spazio di archiviazione che poi si è costretti a pagare anche nel momento in cui nessuno sta utilizzando quella specifica

applicazione. Un altro aspetto positivo è che con questa tecnologia lo sviluppatore può facilmente ampliare o modificare la propria applicazione. Questa, infatti, è caricata sul cloud tramite diverse funzioni composte da poche righe di codice. Lo sviluppo avviene quindi quasi su singoli byte visto che non si tratta di un'applicazione monolitica ma di un pool di funzioni. [5] Ognuna di queste sarà poi richiamata da uno o più eventi come ad esempio una richiesta http. [6]

Ci sono ovviamente anche degli aspetti negativi nell'utilizzo di questa tecnologia. Uno dei primi è sicuramente quello legato al test e al debug del codice. I programmatori non possono infatti vedere cosa accade effettivamente sull'infrastruttura. Non sanno in che modo il codice viene eseguito e che processi genera. In più questo tipo di soluzione non è adatto ad esecuzioni che richiedono un tempo prolungato. Possono infatti esserci delle limitazioni che il provider inserisce sulla durata di una chiamata. In più su esecuzioni lunghe il pagamento a consumo può non essere conveniente. Bisogna considerare anche l'aspetto delle prestazioni. Il codice di una funzione serverless non è costantemente attivo ma potrebbe richiedere un avvio nel caso in cui non venga utilizzato per diverso tempo. Questo processo è chiamato "avvio a freddo". La velocità di esecuzione in questo caso richiede più tempo rispetto ad un "avvio a caldo", cioè la chiamata di una funzione usata molto di frequente. [7] È bene ricordare che, se una funzione non viene usata per diverso tempo torna inattiva e quindi alla chiamata successiva richiederà più tempo per l'esecuzione.

2.3 Servizi AWS

Come detto in precedenza i servizi offerti da Amazon Web Services sono più di 200 e spaziano ad esempio dai database, al calcolo, al machine learning, ai container e all'archiviazione. Di seguito si elencano i servizi utilizzati per la creazione di applicazioni web serverless.

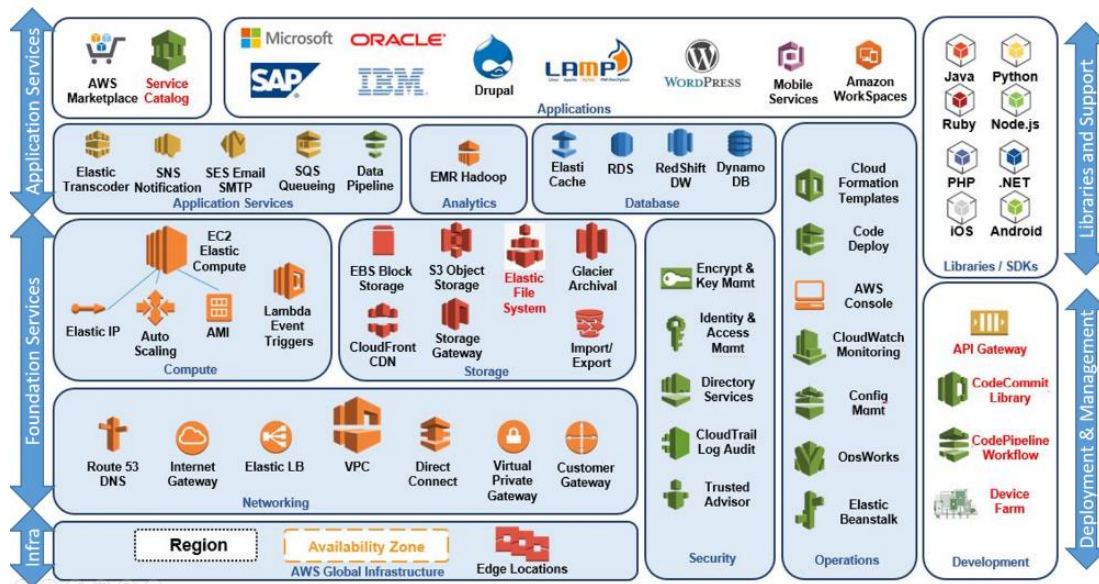


Figura 2 - Servizi AWS⁵

2.3.1 AWS Lambda

Le funzioni lambda permettono di eseguire codice sull'infrastruttura di AWS in base alla ricezione di uno specifico evento. Tra i trigger delle funzioni Lambda ci sono richieste http, modifiche alle tabelle del database, qualche azione specifica eseguita dall'utente o eventi generati da altri servizi AWS. Lambda utilizza la tecnologia serverless per cui lo sviluppatore non deve occuparsi di ciò che riguarda l'esecuzione come, ad esempio, la scalabilità, la tolleranza ai guasti o la manutenzione dei server. Di questo si occuperà il provider. Il suo compito è solo quello di scrivere codice stateless che abbia un preciso scopo. Questo servizio può essere utilizzato per costruire il back-end dell'applicazione web. Se si vuole ad esempio che, nel caso in cui l'utente clicchi il bottone "Aggiungi al carrello" quell'oggetto venga effettivamente aggiunto al carrello, si crea una funzione che si occuperà di eseguire l'operazione e si fa in modo che questa venga evocata al click del bottone. [8]



Figura 3 - Logo AWS Lambda

Il costo di utilizzo di questo servizio dipende dalla durata dell'esecuzione calcolata in millisecondi. Il tempo è calcolato dal momento in cui il codice viene eseguito e il

⁵ <http://awsprocert.com/aws-managed-services/>

conteggio termina quando viene restituito il risultato. Il costo dipende anche dalla quantità di memoria allocata che è anche l'unico parametro settabile insieme all'architettura che può essere x86 o Arm e allo spazio su disco (da 512 MB a 10.240 MB). [9] Le funzioni Lambda hanno anche una limitazione temporale. La loro esecuzione, infatti, non può durare più di 15 minuti. È quindi possibile settare il timeout da un minimo di un secondo ad un massimo di un quarto d'ora. [10]

Per caricare una funzione Lambda è necessario accedere alla console di AWS. A questo punto è possibile scegliere tra tre diverse opzioni. Utilizzare un'immagine del container appoggiandosi quindi ad Amazon Elastic Container Registry⁶. Si tratta di un registro di immagini dei container costituite da un sistema operativo, il codice dell'applicazione e le sue dipendenze. Una seconda opzione è quella di utilizzare dei template già disponibili nella console. Nell'elenco è presente, ad esempio, una semplice funzione hello world ma anche operazione più complesse come funzioni che interagiscono con DynamoDB o che mandano mail quando si clicca un pulsante IoT. La terza opzione permette invece di caricare solamente il codice senza preoccuparsi di eventuali container o sistemi operativi. In questo caso bisognerà scegliere il linguaggio con cui la funzione è stata sviluppata, darle un nome e definire il set di istruzioni (l'architettura) che la eseguirà. Per rendere l'esecuzione ancora più sicura è possibile firmare il codice in modo che Lambda esegua soltanto programmi attendibili.

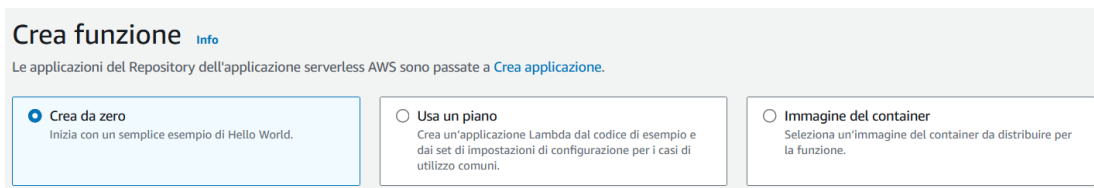


Figura 4 - Creazione di funzione Lambda

Il codice dell'applicazione, per essere caricato, deve essere inserito in un archivio .zip o .jar. In questo file devono essere presenti il codice che verrà eseguito sull'infrastruttura AWS e le relative dipendenze. Il caricamento può essere effettuato sia da Amazon Simple Storage Service sia dal proprio computer.

⁶ <https://aws.amazon.com/it/ecr/>

Di seguito è riportato un esempio, scritto in Java, di funzione Lambda che si occupa di svolgere addizioni, sottrazioni e moltiplicazioni.

```
public class Handler implements RequestHandler<Map<String,String>, Integer> {

    @Override
    public Integer handleRequest(Map<String,String> event, Context context) {
        try {

            int n1 = Integer.parseInt(event.get("n1"));
            int n2 = Integer.parseInt(event.get("n2"));
            String op = event.get("op");
            int res=0;

            switch(op) {
                case "+":
                    res=n1+n2;
                    break;

                case "-":
                    res=n1-n2;
                    break;

                case "*":
                    res=n1*n2;
                    break;

            }

            return res;
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

Figura 5 - Esempio codice funzione Lambda

La classe Handler estende l'interfaccia RequestHandler che richiede due parametri: tipo di input (in questo caso una mappa con chiavi di tipo stringa e valori di tipo stringa) e tipo di output (in questo caso l'oggetto Integer) che devono essere degli oggetti. Con questa interfaccia, a runtime, Java andrà in automatico a deserializzare l'evento nell'oggetto di input. I tipi che possono essere utilizzati sono ad esempio

Integer, Long, String o List<Object>. Per quanto riguarda l'output se questo è un oggetto viene serializzato in una stringa JSON. Se l'oggetto si limita unicamente a fare da wrap ad un tipo primitivo si otterrà solamente la rappresentazione del suo valore. Quando il codice viene chiamato per la prima volta Lambda, che si trova nella fase di

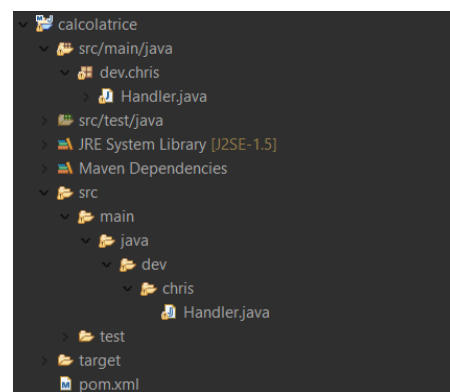


Figura 6 - Struttura del codice

inizializzazione, andrà ad eseguire le parti static del codice e le risorse create rimarranno disponibili in memoria finché la funzione non viene disattivata a causa di una lunga inattività. La funzione `handleRequest` sarà quella che verrà chiamata nel momento in cui si verificherà un evento collegato alla funzione Lambda e accetta una mappa che contiene tutti i parametri legati all'evento. In questo caso si tratta dei due valori sul quale si eseguirà il calcolo e un simbolo che indica l'operazione da compiere. Caricato il codice è necessario modificare il gestore in modo da specificare il nome della funzione da richiamare quando si verifica un evento. Il gestore è composto dal nome del package, dalla classe e dal nome della funzione. Nel caso dell'esempio in questione sarà: `dev.chris.Handler::handleRequest`

Impostazioni di runtime [Info](#)

Runtime
Scegli il linguaggio da utilizzare per scrivere la funzione. L'editor di codice della console supporta solo Node.js, Python e Ruby.

Java 11 ▼ ↺

Nuovo runtime disponibile ✕

È disponibile un nuovo runtime per la lingua della tua funzione: Java 17

Gestore [Info](#)

dev.chris.Handler::handleRequest

Architettura [Info](#)
Scegli l'architettura del set di istruzioni desiderata per il codice della funzione.

☒ x86_64
☐ arm64

Figura 7 - Impostazioni di runtime

Una volta caricato il codice, questo può essere testato utilizzando una specifica tab della console di AWS chiamata: Esegui il test. Per eseguirlo si crea un nuovo evento assegnandogli un nome e dei parametri espressi in formato JSON. L'esecuzione del test restituirà una

JSON dell'evento

```
1 {  
2   "n1": "2",  
3   "n2": "3",  
4   "op": "+",  
5 }
```

Figura 8 - Parametri evento

serie di informazioni come il risultato ottenuto, la durata dell'inizializzazione della funzione e la durata effettiva dell'esecuzione.

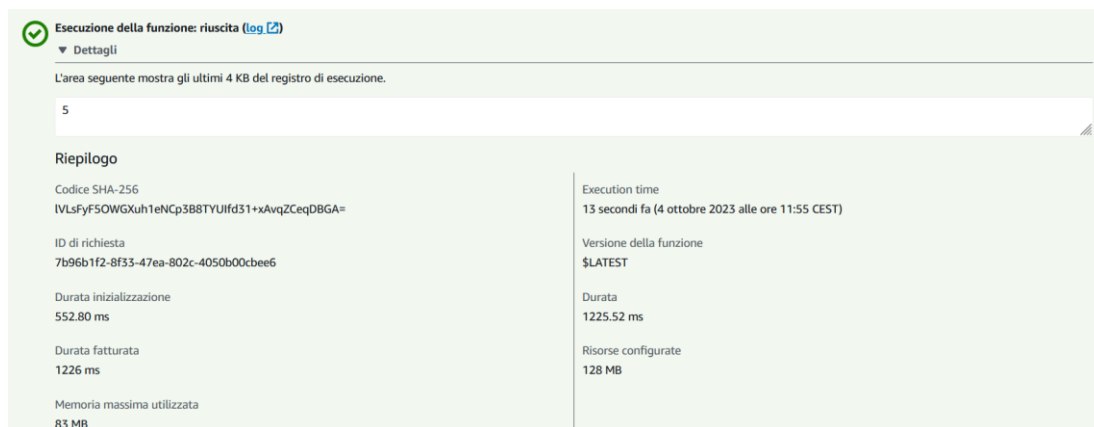


Figura 9 - Risultato di un test di una funzione Lambda

Per ottenere ancora più informazioni sull'esecuzione della funzione Lambda è possibile abilitare la creazione di tracce che vengono analizzate dal servizio X-Ray. Nella tab Configurazione, alla voce Strumenti di monitoraggio e operazioni si possono abilitare vari tipi di tracciamento che raccolgono dati sulle prestazioni, sui valori di utilizzo della CPU, delle memorie, dei dischi o della rete. Una volta attivate le varie opzioni nella tab monitora, se si eseguisse nuovamente il test, dovrebbe apparire una schermata che contiene delle informazioni simili a quelle mostrate di seguito.

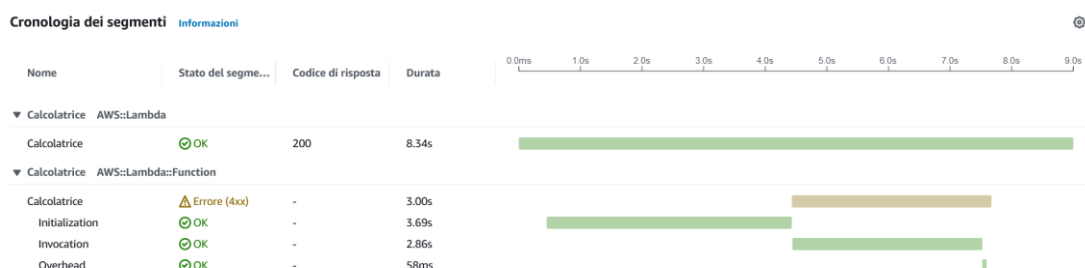
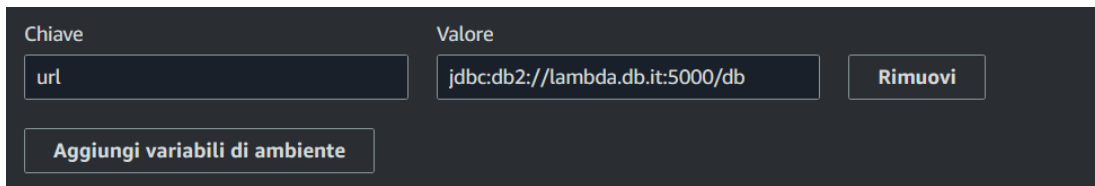


Figura 10 - Monitoraggio X-Ray

Alla funzione Lambda può essere assegnato un url. In questo modo sarà possibile richiamarla utilizzando un browser o qualche altro client http. L'utilizzo di questo endpoint http può essere protetto tramite AWS IAM, in questo modo solo chi è autenticato può eseguire la funzione. È anche possibile non settare delle autorizzazioni aggiungendo magari la logica di autorizzazione nel codice della funzione. All'esecuzione della funzione si possono anche aggiungere delle variabili di ambiente che ne determinino uno specifico comportamento senza dover modificare il codice. Si potrebbero avere due funzioni con lo stesso codice ma con esecuzioni diverse in virtù

di diverse variabili di ambiente. Queste variabili sono definite tramite una coppia chiave valore. Si potrebbero ad esempio utilizzare per indicare l'host di un database.



Chiave	Valore	
url	jdbc:db2://lambda.db.it:5000/db	Rimuovi

Aggiungi variabili di ambiente

Figura 11 - Variabili d'ambiente

Un esempio di riga di codice per recuperare il valore delle variabili d'ambiente è questo: `String urlDb = System.getenv("url");`

2.3.2 AWS IAM

L'AWS Identity e Access Management permette di gestire le autorizzazioni che specifici utenti hanno su precise risorse. Si tratta di un controllore degli accessi. Tutti gli account AWS alla creazione posseggono un utente root che permette di eseguire qualsiasi operazione. Tuttavia, è preferibile utilizzarlo il meno possibile nelle attività che non richiedono quel preciso livello di autorizzazione. In



Figura 12 - Logo AWS Identity and Access Management (IAM)

IAM è possibile creare degli utenti che sono identificati da un nome e da un ARN (Amazon resource name) e al quale è possibile accedere o tramite password o chiavi. Si possono raccogliere gli utenti in gruppi e assegnare specifiche autorizzazioni (tramite policy descritte in formato JSON⁷) sia all'intero gruppo sia al singolo utente. È possibile anche definire dei ruoli che sono simili agli utenti ma non posseggono una password o una chiave d'accesso e soprattutto possono essere assumibili da chiunque. I ruoli sono utili in tutti i software in cui è necessario accedere ai servizi di AWS perché non è necessario inserire nel codice le credenziali che potrebbero essere estratte dagli utenti finali. Vengono utilizzati anche nell'ambiente cloud, ad esempio, se si ha una funzione lambda che deve accedere a S3⁸ le si assegna un ruolo che ha questa autorizzazione in modo che possa eseguire il suo compito. [11]

⁷ <https://www.json.org/json-en.html>

⁸ <https://aws.amazon.com/it/s3/>

Come detto in precedenza una volta creato l'account AWS questo avrà i permessi di

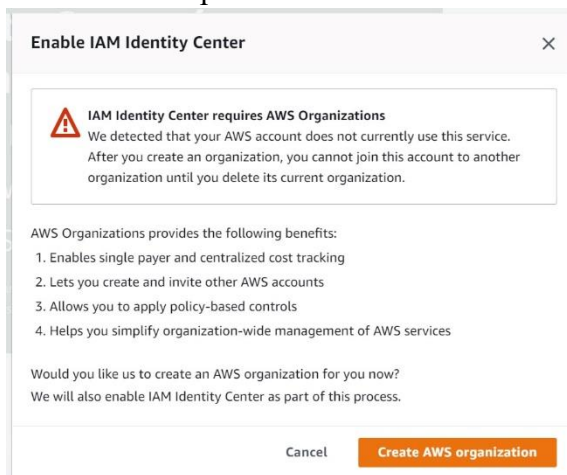


Figura 14 - Creazione di un'organizzazione

root ma è preferibile utilizzare account con livelli di autorizzazione minori. Per aggiungere un utente che abbia le autorizzazioni da amministratore si può utilizzare la console AWS tramite il servizio IAM Identity Center. Per creare account che siano legati alle risorse definite con l'account di root è necessario creare un'organizzazione.

Per farlo si utilizza il servizio AWS

Organizations⁹. L'utente root diventerà l'account di gestione dell'organizzazione. In IAM Identity Center è ora possibile aggiungere un nuovo utente assegnandogli un nome e facendogli recapitare una mail che conterrà le istruzioni per configurare la password e il link nel quale sarà presente il form di login per poter accedere all'account. Il link è

univoco per ogni organizzazione. Per permettere al nuovo utente di operare come un amministratore è necessario aggiungerlo ad un gruppo a cui è assegnato un set di autorizzazioni. In questo

Figura 13 - Set autorizzazioni amministratore

caso è possibile utilizzare un set già creato da Amazon che prende il nome di AdministratorAccess. Ce ne sono ovviamente anche altri che permettono di definire utenti che possano avere accesso solo alle operazioni di pagamento o a quelle legate ai database.

⁹ <https://aws.amazon.com/it/organizations/>

La struttura dell'organizzazione dovrebbe quindi assomigliare a questa:

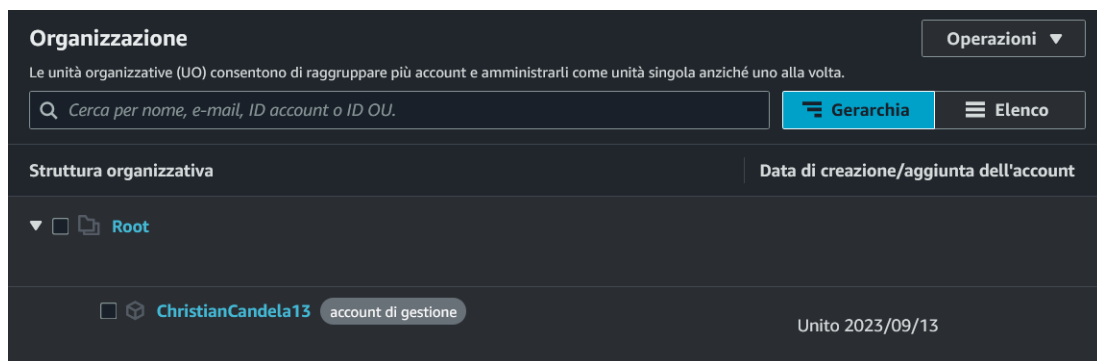


Figura 15 - Struttura organizzazione

Creata l'utente amministratore si può procedere con la definizione di utenti, gruppi e ruoli. Questi elementi possono essere anche creati in automatico quando si utilizzano altri servizi AWS. Un esempio può essere il ruolo che viene aggiunto quando si definisce una funzione Lambda. In questo caso viene

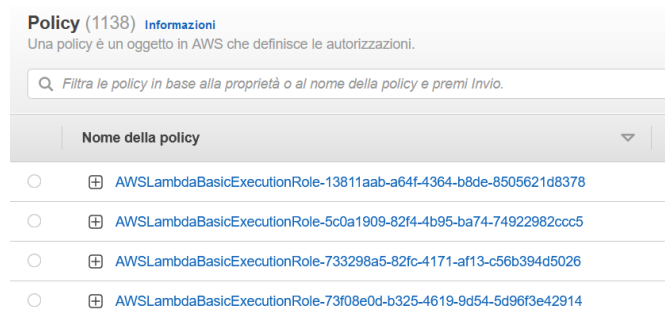


Figura 16 - Policy

creato un ruolo che permette di creare log visibili in CloudWatch¹⁰. Per consentire ad utenti, ruoli o gruppi di compiere operazioni su AWS è necessario assegnargli delle policy. Di seguito un esempio di come sono composte:



Figura 17 - Policy in formato JSON

¹⁰ <https://aws.amazon.com/it/cloudwatch/>

Andando ad analizzare il file si nota che ogni regola è composta da diversi elementi tra cui: effect, action, sid, resource, condition. La prima indica l'effetto della regola che può essere allow, cioè quelle specifiche operazioni sono eseguibili da chi possiede questa policy, oppure deny cioè quelle operazioni non sono eseguibili. Di default ogni azione è settata come deny. Action racchiude invece tutte le chiamate alle API dei servizi AWS che andiamo a controllare con quella regola. Resource rappresenta invece l'ARN della risorsa a cui è applicata la regola. Il sid permette di dare un id ad uno specifico statement per differenziarlo dagli altri. Con condition si descrive la situazione in cui quella policy garantisce i permessi. Ad esempio, è possibile svolgere quelle action solo se è attiva un'autenticazione multi-fattore.

```
"Condition": {"Bool": {"aws:MultiFactorAuthPresent": "true"}}
```

Non è però necessario scrivere a mano il file JSON che le definisce. La console di AWS mette infatti a disposizione un'interfaccia grafica che consente di cercare il servizio desiderato e di selezionare le autorizzazioni che si vogliono concedere.

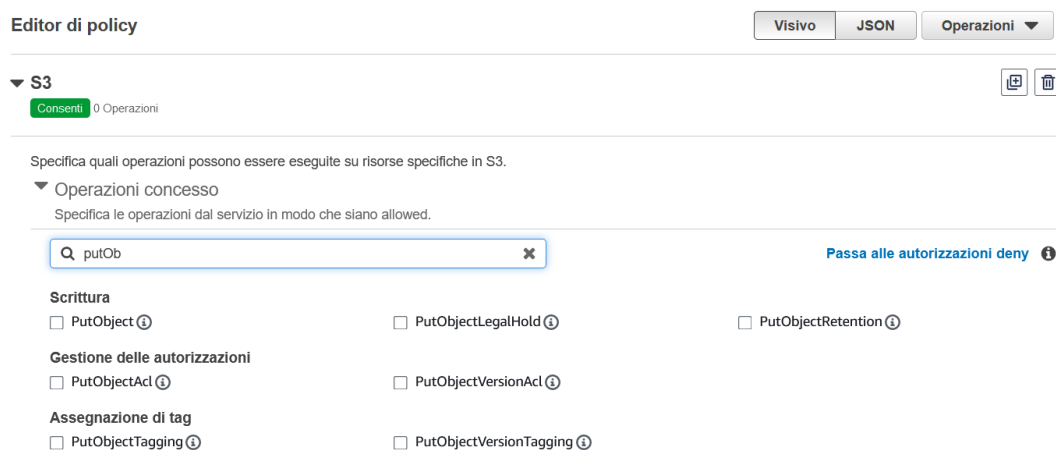


Figura 18 - Interfaccia per creare policy

Per migliorare la gestione delle autorizzazioni IAM mette a disposizione anche un sistema di analisi degli accessi. Questo servizio monitora gli accessi eseguiti dai vari ruoli o utenti alle risorse. Una volta creato un sistema di analisi questo salverà le varie autorizzazioni concesse dall'infrastruttura in modo che siano analizzabili successivamente per capire se ci sono dei privilegi troppo larghi. Per la sicurezza di ogni sistema è infatti necessario perseguire il minimo privilegio, cioè, bisogna concedere solo le autorizzazioni necessario per eseguire quello specifico compito. Con

questo servizio è anche possibile generare automaticamente delle policy per un'applicazione. Basta farla eseguire una singola volta e il sistema di analisi definirà il minor numero di privilegi necessari per il completamento dell'esecuzione. Per evitare di dover analizzare centinaia di voci del tutto legittime è possibile definire dei criteri di archiviazione per eliminare dall'elenco tutti quegli accessi che non minano la sicurezza. [12]

ID risultato	Risorsa	Account de...	Principale ...	Condizione	Condiviso t...	Livello di a...	Agg...
e7a16c77-27fc-4f33-a...	Ruolo IAM aws-reserve...		Utente federato arn:awsiam...	-	-	Write, Taggi...	7 minu...
1669ee0a-938f-4db4-...	Ruolo IAM us-east-1_z...		Utente federato cognito-ide...	Destinatari Cognito us-east-1:631ce6...	-	Write	7 minu...

Figura 19 - Sistema di analisi degli accessi

2.3.3 Amazon API Gateway

Con l'Amazon API Gateway si creano dei punti di accesso alle funzionalità del back-end. Questo servizio permette di definire API che consentono, ad esempio, l'interazione con le funzioni Lambda dall'esterno. Per realizzare la comunicazione si possono utilizzare o le WebSocket o le API RESTful. La prima costituisce una comunicazione bidirezionale e stateful per cui si crea una vera e propria connessione tra client e server. Può essere molto utile per servizi di chat o di streaming di dati. La seconda invece è del tutto stateless ed è fortemente ottimizzata per sfruttare la tecnologia serverless. [13]

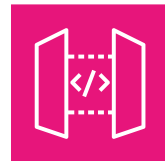


Figura 20 - Logo Amazon API Gateway

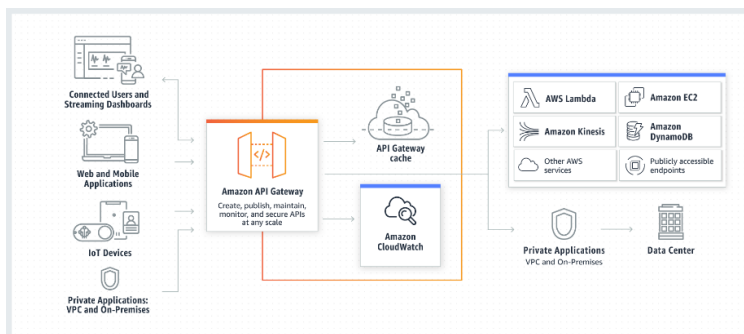


Figura 21 - Architettura Amazon API Gateway

Quando si crea una nuova API è possibile costruirla completamente da zero, clonarne una già esistente nel proprio account AWS, importare API create e condivise da altri utenti o scoprire

alcuni esempi messi a disposizione da Amazon per poter capire meglio il loro funzionamento. Crea l'API bisogna definire il metodo della richiesta che può essere

GET, POST o comunque tutti i metodi supportati dal protocollo http. Definito il metodo bisogna specificare con cosa deve integrarsi questa richiesta: un servizio AWS, una funzione Lambda o un endpoint http già esistente.

Per poter utilizzare l'API, magari da una pagina web, sarà necessario anche abilitare CORS cioè il Cross-Origin Resource Sharing che permette di scaricare risorse che provengono da domini differenti da dove si effettua la richiesta. Per farlo basta selezionare / e poi abilitare CORS. Per rendere accessibile l'API dall'esterno è necessario fare il deploy andando a creare una nuova fase. Si indica

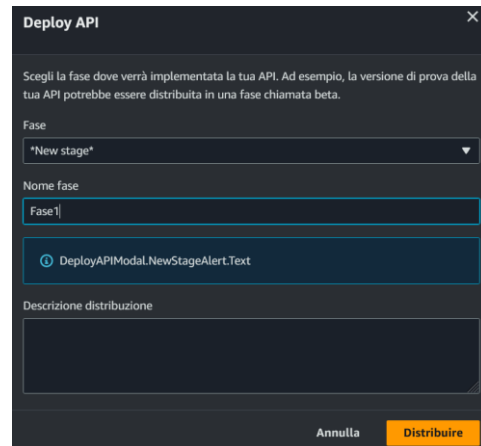


Figura 22 - Deploy API

quindi un nome per questa nuova fase e si clicca su distribuire. A questo punto sarà disponibile un url a cui è possibile mandare delle richieste http interagendo ad esempio con una funzione Lambda. Anche l'API può essere testata e questo può essere fatto all'interno della tab test. Nell'esempio che segue è stata creata una API con metodo POST collegata alla funzione Lambda mostrata nei capitoli precedenti. È quindi possibile mandare i due numeri e l'operazione tramite una richiesta http ottenendo come risposta il risultato.



Figura 23 - Test API

Questa richiesta può essere effettuata anche al di fuori di AWS. Utilizzando questo sito¹¹ che permette di inviare richieste di tipo POST si ottiene lo stesso risultato.

¹¹ <https://reqbin.com/post-online>

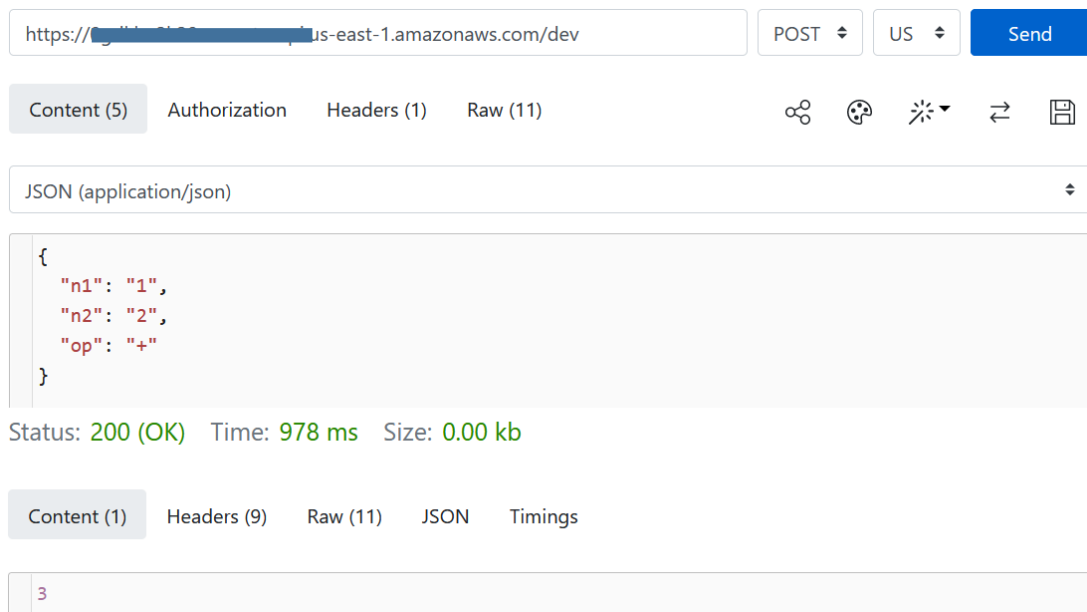


Figura 24 - Richiesta POST

Alle API è anche possibile collegare delle specifiche autorizzazioni in modo che non tutti gli utenti che mandano una richiesta a quell'url possano ottenere un risultato. Nella tab autorizzazioni è quindi possibile definire un controllo degli accessi

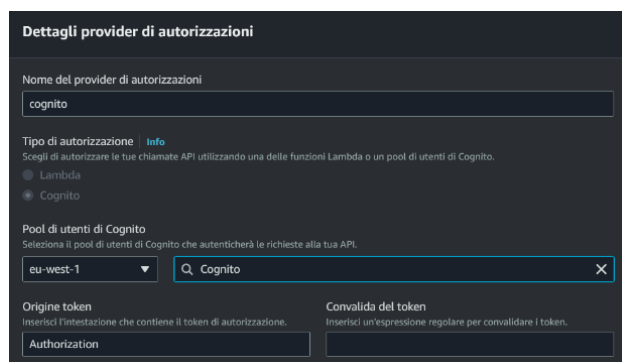


Figura 25 - Provider di autenticazione

che può essere basato su autorizzazione Lambda, che permette di implementare un metodo di autorizzazione personalizzato, oppure può utilizzare un pool di utenti creato in Amazon Cognito. Quando si imposta il provider di autorizzazione bisogna indicare anche il nome dell'header che conterrà il token. Definito il provider bisogna collegarlo all'effettiva richiesta all'API. Si seleziona quindi il metodo http scelto e si modifica la richiesta aggiungendo nella sezione autorizzazione il provider creato precedentemente. Fatta questa modifica è necessario eseguire il deploy dell'API.

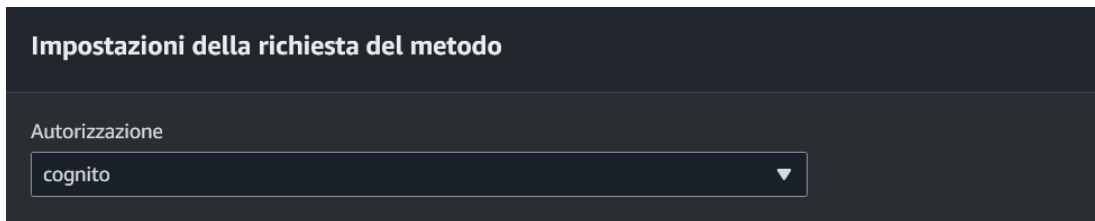


Figura 26 - Definizione dell'autorizzazione dell'API

Provando adesso ad eseguire la richiesta http fatta in precedenza si ottiene un codice di errore 401 (unauthorized).

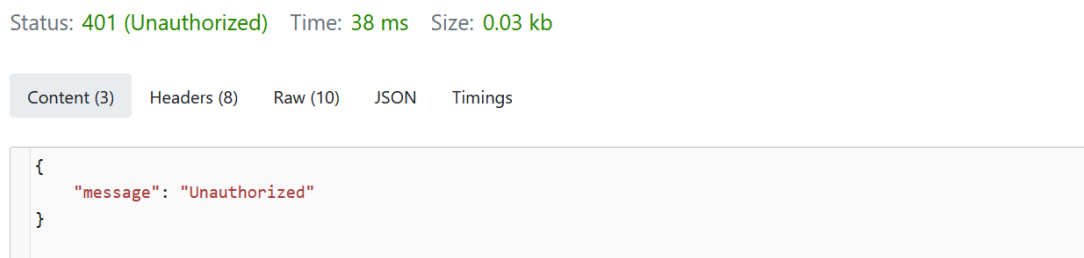


Figura 27 - Accesso negato alla funzione Lambda

Aggiungendo però alla richiesta http il token ottenuto facendo l'autenticazione tramite Cognito la funzione Lambda torna a restituire il risultato corretto. Da notare che il nome dell'header che contiene il token coincide con quello inserito nella creazione del provider nella sezione origine token.

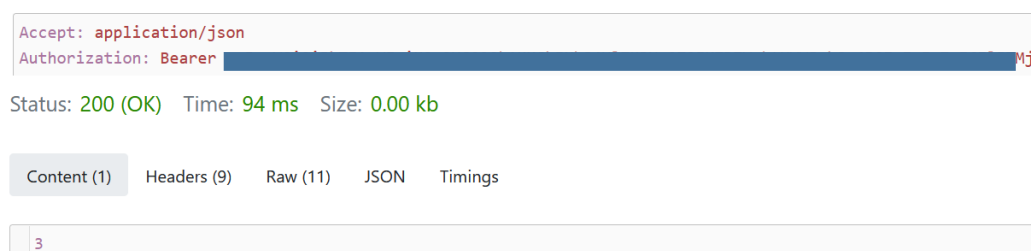


Figura 28 - Richiesta con token

2.3.4 AWS Amplify

AWS Amplify è un servizio con il quale si può costruire sia il front-end sia il back-end di applicazioni web e mobile. È possibile creare un back-end multiplatforma (IOS, Android, Flutter, Web e React) che si occupi dell'autenticazione, della gestione dei dati e dell'archiviazione. AWS Amplify può anche essere utilizzato per l'hosting di una pagina web statica o per caricare del codice, anche tramite repository, in modo da creare delle pagine dinamiche che costituiscano una vera e propria applicazione. Possiamo quindi rendere pubblicamente accessibile una pagina html senza preoccuparci di nessuna impostazione lato server. [14]



Figura 29 - Logo AWS Amplify

Dalla console di AWS si possono scegliere due strade per creare una web app con Amplify. La prima è quella di collegare AWS ad un repository Git in modo da rendere disponibile a livello globale il proprio software. Un'altra strada è quella di creare una app praticamente da zero tramite Amplify Studio. Scegliendo la seconda opzione sarà molto semplice anche hostare una semplice pagina web che utilizza le API Gateway definite in precedenza. Per creare una nuova applicazione oltre all'interfaccia grafica si può utilizzare AWS CLI¹² cioè, l'interfaccia a linea di comando. AWS mette a disposizione anche un IDE cloud-base chiamato Cloud9¹³ che permette di scrivere, eseguire e testare il codice direttamente dal browser. Per creare una nuova app Amplify tramite questo IDE serve per prima cosa installare l'interfaccia CLI.

```
bash - "ip-172-31-2-153.e x Immediate x +
AWSReservedSSO_AdministratorAc :~/environment $ npm install -g @aws-amplify/cli
```

Figura 30 - Installare CLI

In modo da poter successivamente impartire i comandi per la creazione di un nuovo progetto Amplify. Durante questa operazione sarà necessario impostare alcuni parametri come il nome del progetto, la località in cui verrà hostato e il metodo di

¹² <https://aws.amazon.com/it/cli/>

¹³ <https://aws.amazon.com/it/pm/cloud9/>

accesso che può essere ad esempio effettuato con le chiavi di accesso reperibili in AWS IAM. Selezionato o creato l'utente dalla console AWS IAM basterà cliccare su: Crea chiave di accesso e poi dall'elenco presentato a



Figura 31 - Chiavi di accesso IAM

schermo scegliere: Interfaccia a riga di comando (CLI). È bene ricordare che la secretAccessKey viene mostrata una sola volta e chiusa la pagina che permette di visualizzarla non potrà essere più recuperata. La chiave di accesso resta invece visibile.



Figura 32 - Creazione nuovo progetto Amplify

Creato il progetto si possono aggiungere degli elementi effettivamente visibili all'utente come una pagina web. Nella tab Hosting environments, della console AWS, vengono mostrati i vari contenitori da cui è possibile caricare ciò che si vuole hostare sull'infrastruttura. Si possono selezionare diversi sistemi di controllo delle versioni ma è anche possibile caricare direttamente dei file senza usare Git. Se si sceglie questa opzione si possono recuperare file da S3 o caricarli direttamente dal computer locale. In questo caso i file devono essere inseriti in un .zip.

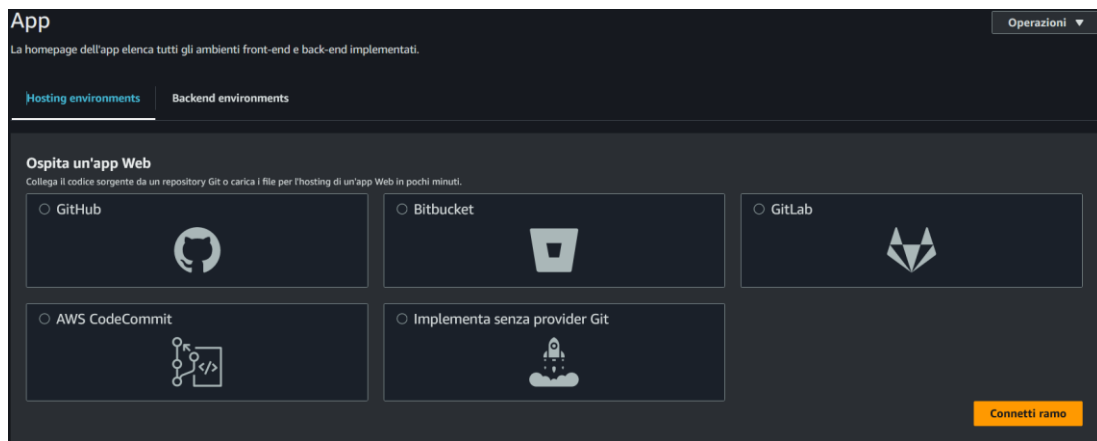


Figura 33 - Hosting environments

Come esempio verrà caricato un file html che contiene un form nel quale è possibile inserire due numeri e selezionare un'operazione.

```

<html>
  <head>
    <title>Calcolatrice</title>
  </head>
  <body>
    <form>
      <input type="text" id="n1" name="n1">
      <select name="op" id="op">
        <option value="+>+</option>
        <option value="->-</option>
        <option value="*>*</option>
      </select>
      <input type="text" id="n2" name="n2">
    </form>
    <button onclick="richiestaAJAX()" ">Calcola</button>
  </body>
</html>

```

Figura 34 - Esempio pagine html

Cliccando il bottone si richiama l'API creata nei capitoli precedenti tramite una richiesta AJAX.

```

function richiestaAJAX() {
  var theXhr = myGetXmlHttpRequest();
  theXhr.onreadystatechange = function() { callback(theXhr); };

  var uri = "https://[redacted].us-east-1.amazonaws.com/dev";

  try {
    theXhr.open("post", uri, true);
  }
  catch(e) {
    alert(e);
  }

  theXhr.setRequestHeader("connection", "close");
  theXhr.setRequestHeader("Content-Type", "application/json");

  var req = new Object();
  req.n1=document.getElementById("n1").value;
  req.n2=document.getElementById("n2").value;
  req.op=document.getElementById("op").value;

  theXhr.send(JSON.stringify(req));
}

```

Figura 35 - Richiesta AJAX

La pagina web caricata su Amplify appare quindi così e una volta premuto il bottone calcola il risultato che sarà mostrato in un pop up.

Figura 36 - Applicazione Amplify

Attraverso l'interfaccia CLI possiamo aggiungere diversi elementi al back-end dell'applicazione Amplify come ad esempio le API. Con il comando: `amplify add api`, il sistema ci permette di crearne una di tipo REST o GraphQL, di assegnargli un nome,

un indirizzo e associargli una funzione Lambda. Tutti i file creati possono poi essere modificati attraverso l'IDE. Per caricare queste modifiche sul cloud sarà necessario utilizzare il comando: `amplify push`.

2.3.5 Amazon Cognito

Amazon Cognito è un servizio che permette la gestione delle identità degli utenti all'interno dell'applicazione. Con Cognito si permette agli utilizzatori dell'app di iscriversi o di accedere agli account. [15] Con questa funzionalità è possibile creare sia un pool di utenti sia un pool di identità. Per quanto riguarda il primo, questo crea una directory di utenti



Figura 37 - Logo Amazon Cognito

che possono autenticarsi ed essere autorizzati ad eseguire specifici compiti all'interno dell'applicazione. È anche possibili registrarsi ed autenticarsi tramite servizi terzi come Facebook o Google. Se si vuole aumentare il livello di sicurezza del proprio account è possibile anche abilitare l'autenticazione a più fattori. Il pool di identità viene invece utilizzato nel momento in cui l'utente deve accedere alle risorse di AWS. [16]

Per utilizzare le funzionalità di Cognito è quindi necessario creare un bacino di utenti. Durante la creazione bisogna specificare con quali elementi è possibile fare l'accesso: nome utente, mail o numero di telefono. È necessario anche indicare le regole per la creazione della password: quanti numeri, lettere maiuscole, lettere minuscole o caratteri speciali deve contenere. Esistono delle impostazioni già definite da Amazon Cognito ma possono anche essere create delle policy totalmente da zero. Per aumentare la sicurezza degli account c'è la possibilità di abilitare un'autenticazione a più fattori sia tramite SMS sia tramite delle app di autenticazione come Google Authenticator. Per creare un pool di utenti bisogna anche avere un indirizzo mail verificato che possa mandare messaggi alle persone che vogliono registrarsi. Per

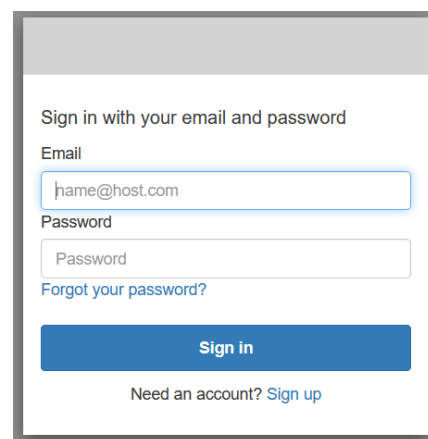


Figura 38 - Interfaccia login

verificare un indirizzo si utilizza il servizio Amazon Simple Email Service¹⁴. È possibile anche servirsi di un indirizzo mail Cognito di default che però permette di inviare un massimo di 50 messaggi al giorno. Cognito permette anche di creare una pagina ospitata che consente agli utenti di registrarsi e di effettuare il login. Quando si configura questa pagina è necessario specificare se si ha intenzione di utilizzare un dominio Cognito che avrà quindi la dicitura: XXXX.auth.eu-west-1.amazonaws.com o se si vuole usare un dominio che si è acquistato in precedenza. Bisogna poi aggiungere un url di callback che indica la pagina a cui si vuole reindirizzare l'utente una volta che l'autenticazione ha avuto successo. Anche la gestione del token di autorizzazione va specificata: si può decidere se inserirlo all'interno dell'url una volta effettuato l'accesso o se si vuole inviare nella risposta http nel momento in cui il login è stato completato correttamente.

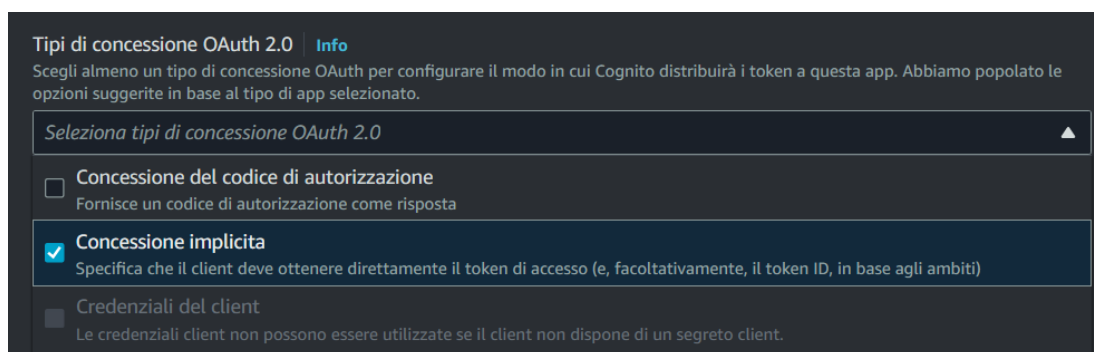


Figura 39 - Definire come Cognito distribuisce il token

Definito il pool di utenti è possibile popolarlo direttamente dalla console AWS. Per farlo basta cliccare il bottone: Crea utente, specificare l'indirizzo mail da utilizzare al momento del login e la password, che può anche essere generata automaticamente. Questa password provvisoria dovrà poi essere modificata una volta che quello specifico utente ha eseguito il login.

¹⁴<https://aws.amazon.com/it/ses/>

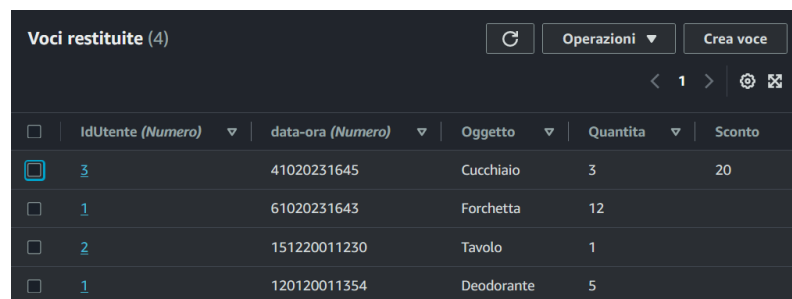
2.3.6 Amazon DynamoDB

Amazon DynamoDB è un database non relazionale completamente gestito questo vuol dire che si tratta di una base di dati noSQL. A differenza dei database SQL, quelli noSQL non richiedono uno schema ben definito ma soltanto una chiave primaria per ogni tabella. Anche questo servizio come gli altri disponibili su AWS non richiede di occuparsi della gestione vera e propria del database come, ad esempio, la creazione di backup o l'aumento delle risorse nel caso in cui aumenti il numero di utenti che utilizzano l'applicazione. Un altro aspetto positivo di utilizzare DynamoDB è che essendo un servizio web le richieste sono tutte stateless e non è quindi necessario instaurare e mantenere aperte connessioni come invece avviene con i RDBMS. Trattandosi di un database non relazionale non è possibile utilizzare join e le tabelle sono lette una alla volta. [17]



Figura 40 - Logo Amazon DynamoDB

Nella console di AWS si possono creare delle tabelle specificandone il nome e la chiave primaria. Quest'ultima si divide in due elementi: chiave di partizione e chiave di ordinamento che possono essere di tipo numerico, stringa o binario. La prima rappresenta un modo per partizionare dati che contengono informazioni collegate. Se prendiamo come esempio gli ordini effettuati da un cliente e indichiamo il codice del cliente come chiave di partizione tutti gli ordini connessi allo stesso utente saranno salvati nello stesso nodo. Questa strategia permette di avere prestazioni migliori se si progettano correttamente le chiavi di partizione. Bisogna preferire delle chiavi di partizione che possono assumere molti valori in modo da distribuire meglio il carico tra le partizioni. La chiave di ordinamento ha invece lo scopo di ordinare gli elementi che si trovano in una partizione. Questa è opzionale e deve



Voci restituite (4)						Operazioni	Crea voce
☐	IdUtente (Numero)	data-ora (Numero)	Oggetto	Quantita	Sconto		
☒	3	41020231645	Cucchiaio	3	20		
☐	1	61020231643	Forchetta	12			
☐	2	151220011230	Tavolo	1			
☐	1	120120011354	Deodorante	5			

Figura 41 - Tabella DynamoDB

essere univoca per singola partizione. Potrebbe ad esempio essere la data e l'ora in cui un utente ha effettuato un acquisto. Se non si specifica una chiave di ordinamento gli

elementi all'interno della partizione non saranno ordinati. [18] Le tabelle possono anche avere degli indici secondari che permettono di accedere più rapidamente agli item. Ad ogni indice secondario viene associata una tabella che può contenere o soltanto la chiave di partizione e quella di ordinamento o tutti gli attributi. Sempre tramite la console si possono visualizzare ed inserire elementi nella tabella. Ogni item può essere rappresentato attraverso un file in formato JSON.

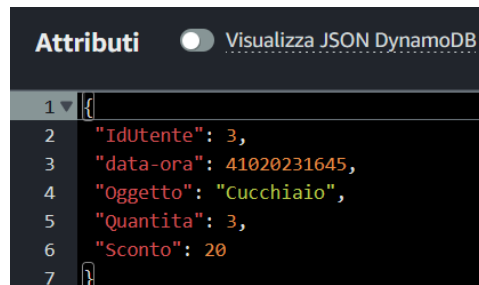


Figura 42 - Rappresentazione in JSON di un item

L'interfaccia grafica permette anche di eseguire delle query sui valori inseriti nella tabella. Si può quindi cercare in base ad un valore specifico di un attributo di partizione e a dei valori uguali, più piccoli o più grandi del valore della chiave di ordinamento. Ovviamente anche i valori degli attributi possono essere dei filtri per la ricerca nella tabella.

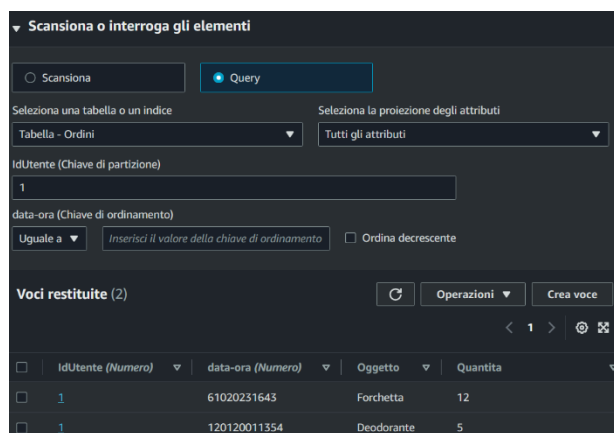


Figura 43 - Query in DynamoDB

Sulle tabelle si può ovviamente operare anche tramite altri servizi AWS come ad esempio le funzioni Lambda. Di seguito è mostrato il codice di una classe che aggiunge un item ad una tabella.

```

public class Handler implements RequestHandler<Void, Void> {

    HashMap<String,AttributeValue> itemValues = new HashMap<String,AttributeValue>();

    @Override
    public Void handleRequest(Void event, Context context) {

        try {

            itemValues.put("IdUtente", AttributeValue.builder().n("1").build());
            itemValues.put("data-ora", AttributeValue.builder().n("061020231724").build());
            itemValues.put("Oggetto", AttributeValue.builder().s("Bicchiere").build());
            itemValues.put("Quantita", AttributeValue.builder().n("2").build());

            PutItemRequest request = PutItemRequest.builder()
                .tableName("Ordini")
                .item(itemValues)
                .build();

            try {

                DynamoDbClient ddb = DynamoDbClient.builder().build();

                ddb.putItem(request);
                System.out.println("Ordini" + " was successfully updated");

            } catch (ResourceNotFoundException e) {
                System.err.format("Error: The Amazon DynamoDB table \"%s\" can't be found.\n");
                System.err.println("Be sure that it exists and that you've typed its name correctly!");
                System.exit(1);
            } catch (DynamoDbException e) {
                System.err.println(e.getMessage());
                System.exit(1);
            }

            return null;
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}

```

Figura 44 - Codice Java che aggiunge item ad una tabella

Per aggiungere un nuovo item è necessario creare una mappa che contenga i nomi delle chiavi e degli attributi con i loro specifici valori. La tabella si popola creando degli oggetti di tipo `AttributeValue`. Il metodo `s` indica che stiamo inserendo un attributo di tipo stringa mentre `n` indica che è di tipo numerico. Per poter poi caricare l'elemento sul cloud bisogna istanziare un oggetto client che si occuperà di fare la richiesta al servizio AWS.

L'esempio seguente mostra invece come sia possibile eliminare un item specificando la chiave di partizione e quella di ordinamento.


```

public class Handler implements RequestHandler<Void, Void> {

    HashMap<String, AttributeValue> itemValues = new HashMap<String, AttributeValue>();

    @Override
    public Void handleRequest(Void event, Context context) {

        Map<String, AttributeValue> keysMap = new HashMap<>();
        keysMap.put("IdUtente", AttributeValue.builder().n("1").build());
        keysMap.put("data-ora", AttributeValue.builder().n("61020231643").build());

        DeleteItemRequest deleteReq = DeleteItemRequest.builder()
            .tableName("Ordini")
            .key(keysMap)
            .build();

        DynamoDbClient ddb = DynamoDbClient.builder().build();
        try {
            ddb.deleteItem(deleteReq);
            return null;
        } catch (DynamoDbException e) {
            System.err.println(e.getMessage());
            System.exit(1);
            return null;
        }
    }
}

```

Figura 45 - Eliminare item da una tabella di DynamoDB

Per svolgere queste operazioni sul DB è ovviamente necessario definire una specifica policy da assegnare al ruolo che assume la funzione quando viene eseguita. In questo modo l'esecuzione avverrà senza errori. La policy qui a fianco specifica che il codice può inserire ed eliminare elementi, aggiornarli ed eseguire sia delle query sia delle scan su una specifica tabella indicata dall'arn.

Editor di policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "VisualEditor0",
6       "Effect": "Allow",
7       "Action": [
8         "dynamodb:PutItem",
9         "dynamodb:DeleteItem",
10        "dynamodb:GetItem",
11        "dynamodb:Scan",
12        "dynamodb:Query",
13        "dynamodb:UpdateItem"
14      ],
15       "Resource": "arn:aws:dynamodb:us-east-1:
16     }
17   ]
18 }

```

Figura 46 - Policy per DynamoDB

2.3.7 AWS X-Ray

AWS X-Ray è un servizio che si occupa di raccogliere tutti i dati legati alle richieste effettuate ad un'applicazione. Questi dati possono essere analizzati per scoprire se sono presenti dei problemi durante l'esecuzione o se è possibile apportare modifiche al proprio software per migliorare l'ottimizzazione. Tra i dati disponibili ci sono quelli sulle richieste, sulle risposte ma anche tutte le chiamate intermedie che vengono effettuate ad altri servizi di AWS. X-Ray funziona tramite tracce che contengono informazioni di specifiche richieste. Queste tracce sono inviate prima ad un processo demone, invocato dal servizio che stiamo monitorando, che si occuperà di caricarli effettivamente nel servizio X-Ray. [19] Possiamo utilizzare questa funzionalità anche nelle funzioni Lambda in modo da avere informazioni, ad esempio, sulla fase di inizializzazione (fase di init dell'ambiente di esecuzione) o quella di invocazione.



Figura 47 - Logo AWS X-Ray

Tornando ad analizzare la funzione Lambda e l'API Gateway definite in precedenza se la creazione di tracce di AWS X-Ray è stata abilitata è possibile visualizzare in maniera precisa quali servizi ha attraversato la richiesta dell'utente.

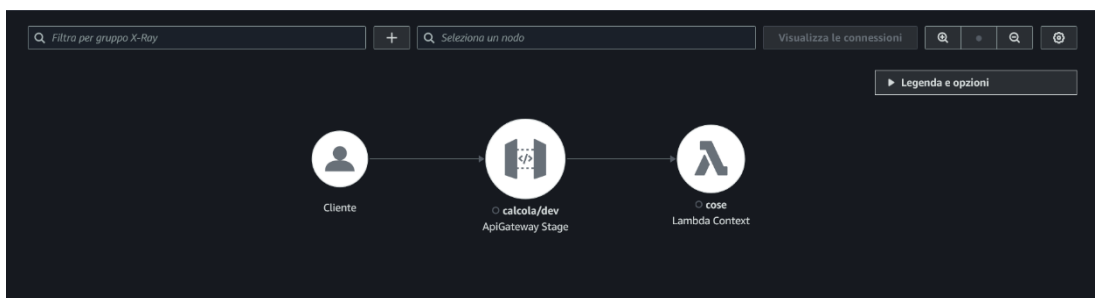


Figura 48 - Traccia AWS X-Ray

Cliccando sui vari tondini si possono visualizzare le varie informazioni delle specifiche chiamate tra cui anche gli errori avvenuti durante l'esecuzione. Questo può tornare molto utile nella fase di debug.

Capitolo 3

3.1 Applicazione

Descritto in maniera generale il funzionamento dei vari prodotti messi a disposizione da Amazon Web Services veniamo ora a un esempio dimostrativo, cioè alla realizzazione di un'applicazione web serverless. Per svilupparla è stato utilizzato il linguaggio Java per il back-end e Javascript per il front-end. L'obiettivo è quello di mostrare come i servizi AWS cooperino all'interno di un progetto più complesso rispetto agli esempi mostrati nel capitolo precedente. Verranno poi presentati dei grafici relativi alle performance dell'app per mostrare visivamente gli aspetti negativi e positivi elencati precedentemente.

3.2 Abstract

L'applicazione realizzata è una bacheca online nel quale è possibile inserire messaggi visualizzabili però soltanto da specifici utenti. I messaggi, infatti, sono crittati tramite chiavi asimmetriche. Nella home vengono visualizzate due tabelle: la prima elenca i vari utenti registrati insieme alle loro chiavi pubbliche, la seconda mostra invece i messaggi inseriti nel sistema insieme ad altre informazioni come il mittente, il destinatario e se sono stati letti o meno. Dalla home è possibile passare ad altre due pagine: login e registrazione. La prima permette di accedere all'area riservata mentre la seconda di creare un nuovo account. Entrambe fanno uso del servizio Amazon Cognito. Effettuato l'accesso nell'area personale verrà mostrata la propria chiave pubblica. Se quest'ultima non è stata generata apparirà un pulsante che permetterà di crearla. Al momento della creazione verrà mostrata anche la chiave privata che andrà opportunamente salvata perché non potrà più essere visionata. Dall'area personale è possibile accedere alla pagina di crittazione e a quella di decrittazione. Nella prima si può aggiungere un messaggio alla bacheca selezionando gli utenti che potranno leggerlo. In questo caso il testo inserito sarà crittato con le chiavi pubbliche dei destinatari. Nella seconda pagina invece saranno visualizzati i messaggi aggiunti alla

bacheca e destinati allo specifico utente che ha fatto il login. Una tabella mostra i messaggi disponibili. In questa sezione dell'applicazione si seleziona quindi uno dei messaggi, si inserisce la chiave privata e si esegue la decrittazione. Una volta che il messaggio sarà stato decrittato verrà indicato come letto.

3.3 Implementazione

3.3.1 Registrazione

Nella console di AWS è stato creato un bacino di utenti in Amazon Cognito che nella registrazione richiede di inserire un username, una password e la mail. Quest'ultima servirà per poter inviare all'utente un link di conferma per attivare l'account. Per far ciò è bene selezionare, durante la creazione del pool, la casella che permette a Cognito di inviare automaticamente messaggi agli utenti. Per personalizzare il messaggio in modo che non invii un codice ma un link bisogna recarsi nella sezione Messaggistica. In basso si trovano i messaggi automatici inviati dal servizio. Tra questi si seleziona messaggio di verifica e poi Link.

Tipo di verifica
Scegli se Cognito invia all'utente un codice che deve immettere o un link su cui deve fare clic per verificare il relativo attributo di contatto.

☐ Code
☒ Link

Oggetto dell'e-mail [Ripristina impostazione di default](#)
Your verification link

Messaggio dell'e-mail [Ripristina impostazione di default](#)
Please click the link below to verify your email address. {##Verify Email##}

Figura 49 - Messaggio di verifica

Per quanto riguarda il codice della funzione Lambda che si occupa del back-end questo può essere diviso in tre specifiche parti. Nella prima si crea un client di Amazon Cognito al quale è possibile inviare le richieste. Per farlo è necessario selezionare la regione in cui è presente il pool di utenti sul quale si vuole operare. Nella seconda parte vengono invece estratti i dati inviati nella richiesta http. In questo caso si tratta della

mail, della password e dell'username. Nella terza parte si va invece a costruire la richiesta specificando l'id del client che si trova nella sezione Integrazione app della console di AWS. Infine, viene fatta la richiesta tramite il client creato precedentemente.

```
public String handleRequest(Map<String,String> map, Context context) {
    // Configura il client di Amazon Cognito
    CognitoIdentityProviderClient cognitoClient = CognitoIdentityProviderClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

    // Estrai i dati dalla richiesta
    String username = (String) map.get("username");
    String password = (String) map.get("password");
    String email = (String) map.get("email");

    // Crea una richiesta di registrazione
    SignUpRequest signUpRequest = SignUpRequest.builder()
        .clientId("4...")
        .username(username)
        .password(password)
        .userAttributes(AttributeType.builder()
            .name("email")
            .value(email)
            .build())
        .build();

    try {
        // Esegui la registrazione
        SignUpResponse response = cognitoClient.signUp(signUpRequest);
        // Se la registrazione ha avuto successo
        return "Registrazione riuscita, controlla la tua mail per attivare l'account ";
    } catch (Exception e) {
        // Errore
        return "Errore durante la registrazione: " + e.getMessage();
    }
}
```

Figura 50 - Codice registrazione

La funzione Lambda per operare ha ovviamente necessità di avere delle specifiche autorizzazioni che posso essere riassunte in questo file JSON.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "VisualEditor0",
            "Effect": "Allow",
            "Action": [
                "cognito-idp:AdminInitiateAuth",
                "lambda:InvokeFunction",
                "cognito-idp:AdminCreateUser",
                "cognito-idp:AdminSetUserPassword",
                "cognito-idp:AdminRespondToAuthChallenge"
            ],
            "Resource": "*"
        }
    ]
}
```

Figura 51 - Policy registrazione

Per accedere alla funzione dall'esterno è stata creata una nuova API con il servizio API Gateway. La pagina html che la utilizza è stata caricata su AWS Amplify. Per farlo è stata prima creata una nuova applicazione e poi nella sezione Hosting Environments è stato aggiunto un nuovo ambiente nel quale è stato caricato un file zip contenente tutte le pagine dell'applicazione. Nella pagina web (registrazione.html) sono presenti le caselle di testo dove inserire i dati necessari e un pulsante che una volta cliccato fa una richiesta Ajax all'API. A fianco è possibile vedere come appare la pagina di registrazione.

Figura 52 - Pagina di registrazione

3.3.2 Login

Per gestire il login bisogna impostare nella console di Cognito, precisamente nella sezione Client dell'app, il flusso di autenticazione ALLOW_ADMIN_USER_PASSWORD_AUTH in modo che sia possibile accedere agli account del pool utilizzando lo username e la password. Si indicherà questo flusso di autenticazione anche nel codice Java della funzione Lambda.

Anche in questo caso il codice inizia con la creazione di un client Cognito e recupera i dati dalla richiesta post effettuate tramite un API Gateway. Per effettuare la richiesta di autenticazione bisogna creare una mappa che contenga l'username e la password e costruire poi l'effettiva richiesta avendo cura di specificare il flusso di autenticazione (quello indicato al paragrafo precedente), l'id del pool utenti e l'id del client applicativo. Eseguita la richiesta, il contenuto della risposta viene inserito in un bean chiamato Risultato. Questo oggetto contiene una stringa che indica se l'operazione è andata a buon fine o se è fallita e il token di

```
public class Risultato {

    private String result;
    private String idToken;
    private String messaggio;
    public String getResult() {
        return result;
    }
    public void setResult(String result) {
        this.result = result;
    }
    public String getIdToken() {
        return idToken;
    }
    public void setIdToken(String idToken) {
        this.idToken = idToken;
    }
    public String getMessaggio() {
        return messaggio;
    }
    public void setMessaggio(String messaggio) {
        this.messaggio = messaggio;
    }
}
```

Figura 53 - Bean risultato

autenticazione. Questo bean sarà automaticamente trasformato in una stringa JSON che verrà interpretata dal codice Javascript presente nella pagina html. La funzione Lambda in questo caso avrà bisogno delle seguenti policy su cognito-idp: Admin Initiate Auth, Admin Respond To Auth Challenge, Admin Update User Attributes, Initiate Auth e Respond To Auth Challenge.

```
public Risultato handleRequest(Map<String,String> map, Context context) {
    // Configura il client di Amazon Cognito
    CognitoIdentityProviderClient cognitoClient = CognitoIdentityProviderClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

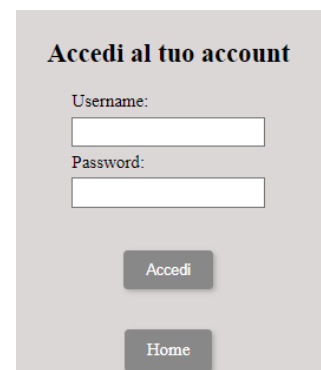
    // Estrai i dati dalla richiesta
    String username = (String) map.get("username");
    String password = (String) map.get("password");

    // Crea una richiesta di registrazione
    Risultato ris = new Risultato();
    Map<String, String> authParameters = new HashMap<>();
    authParameters.put("USERNAME", username);
    authParameters.put("PASSWORD", password);
    AdminInitiateAuthRequest authRequest = AdminInitiateAuthRequest.builder()
        .authFlow(AuthFlowType.ADMIN_USER_PASSWORD_AUTH)
        .userPoolId("eu-north-1_XXXXXXXXXXXX")
        .clientId("XXXXXXXXXXXX")
        .authParameters(authParameters)
        .build();

    try {
        AdminInitiateAuthResponse authResponse = cognitoClient.adminInitiateAuth(authRequest);
        String idToken = authResponse.authenticationResult().idToken();
        ris.setIdToken(idToken);
        ris.setResult("Ok");
        ris.setMessaggio("Ok");
        return ris;
    } catch (Exception e) {
        ris.setResult("Fail");
        ris.setMessaggio(e.toString());
        ris.setIdToken("Error");
        return ris;
    }
}
```

Figura 54 - Codice Login

Nella pagina html nel caso in cui il login abbia avuto esito positivo vengono creati due cookie: uno conterrà l'username della persona che ha effettuato il login l'altro invece conterrà il token. Entrambi avranno una validità di un'ora. Se il login ha successo c'è un indirizzamento automatico alla home se invece fallisce un messaggio di errore viene inserito sotto il pulsante Accedi.



The image shows a login form with the title "Accedi al tuo account". It contains two text input fields, one labeled "Username:" and the other "Password:". Below these fields are two buttons: "Accedi" and "Home". The form is set against a light gray background.

Figura 55 - Pagina Login

3.3.3 Area personale

L'area personale di un utente può assumere diverse forme in base alle informazioni presenti nel suo account. La prima volta che effettua il login apparirà così:



Figura 56 - Area personale prima di generare chiave pubblica

Quando si accede all'area personale viene chiamata una funzione Lambda che si occupa di cercare in DynamoDB se è disponibile o meno una chiave pubblica per quello specifico utente. Nel caso in cui non si trovi nessuna corrispondenza la funzione lo comunica al front-end che chiede all'utente di generare una coppia di chiavi asimmetriche.

```
if(risposta.result=="Ok"){
    messaggio.innerHTML = "<h2>La tua chiave pubblica: </h2><textarea>" + risposta.chiave+"</textarea>";
}else{
    messaggio.innerHTML = "<h2>Non hai ancora generato la tua chiave pubblica</h2>";
    var pulsante = document.createElement("button");
    pulsante.innerText = "Genera";
    pulsante.setAttribute("action", "richiestaAJAXCrea()");
    pulsante.addEventListener("click", function() {
        richiestaAJAXCrea();
    });
    var contenitore = document.getElementById("chiave");
    contenitore.appendChild(pulsante);
}
```

Figura 57 - Codice JS che mostra o la chiave pubblica o chiede all'utente di generarla

In DynamoDB è ovviamente stata creata una tabella chiamata chiavi con chiave di ripartizione che contiene l'username dell'utente.


```

public Risultato handleRequest(Map<String,String> map, Context context) {
    // Configura il client di Amazon Cognito
    DynamoDbClient dynamoDb = DynamoDbClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

    String username = (String) map.get("username");
    // Specifica i parametri per la query
    QueryRequest queryRequest = QueryRequest.builder()
        .tableName("chiavi")
        .keyConditionExpression("username = :val")
        .expressionAttributeValues(Collections.singletonMap(":val", AttributeValue.builder().s(username).build()))
        .build();

    // Esegui la query
    QueryResponse response = dynamoDb.query(queryRequest);
    dynamoDb.close();
    // Elabora i risultati
    Risultato ris = new Risultato();
    for (Map<String, AttributeValue> item : response.items()) {
        // Elabora ciascun elemento restituito
        ris.setChiave(item.get("chiave").s());
        ris.setRisult("Ok");
        return ris;
    }
    ris.setChiave("Error");
    ris.setRisult("Fail");
    return ris;
}

```

Figura 58 - Codice per la ricerca delle chiavi pubbliche

Nel codice viene quindi eseguita una query che permette di ottenere l'item contenente la chiave pubblica di quello specifico utente. Il risultato viene restituito tramite un bean che conterrà la chiave e il risultato dell'operazione. Nel caso in cui non si trovi nessuna chiave verrà restituito il risultato fail. In quel caso la pagina html mostrerà il pulsante che permette di generare le chiavi utilizzando un'altra funzione Lambda.

```

public Risultato handleRequest(Map<String,String> map, Context context) {
    Risultato result = new Risultato();
    PublicKey publicKey;
    PrivateKey privateKey;
    KeyPairGenerator keyPairGenerator;
    try {
        keyPairGenerator = KeyPairGenerator.getInstance("RSA");
        keyPairGenerator.initialize(2048); // Key size (2048 bits is common)
        KeyPair keyPair = keyPairGenerator.generateKeyPair();
        publicKey = keyPair.getPublic();
        privateKey = keyPair.getPrivate();
        result.setPrivata(Base64.getMimeEncoder().encodeToString(privateKey.getEncoded()).replaceAll("\\r|\\n", ""));
        result.setPubblica(Base64.getMimeEncoder().encodeToString(publicKey.getEncoded()).replaceAll("\\r|\\n", ""));
        result.setResult("Ok");
    } catch (NoSuchAlgorithmException e) {
        // TODO Auto-generated catch block
        result.setResult("Fail");
        result.setPrivata("Fail creazione chiave " + e.toString());
        result.setPubblica("Fail");
        return result;
    }

    Map<String, AttributeValue> valori = new HashMap<>();
    String username = (String) map.get("username");
    valori.put("username", AttributeValue.builder().s(username).build());
    valori.put("chiave", AttributeValue.builder().s(result.getPubblica().replaceAll("\\r|\\n", "")).build());
    PutItemRequest request = PutItemRequest.builder()
        .tableName("chiavi")
        .item(valori)
        .build();

    try {
        DynamoDbClient ddb = DynamoDbClient.builder().build();
        ddb.putItem(request);
    } catch (Exception e) {
        result.setResult("Fail");
        result.setPrivata("Fail salvataggio in database " + e.toString());
        result.setPubblica("Fail");
        return result;
    }
    return result;
}

```

Figura 59 - Codice generazione chiavi asimmetriche

Il codice di questa funzione Lambda genera quindi delle chiavi da 2048 bit. Entrambe le chiavi vengono codificate in base64 ed inviate all'utente che le ha generate. Solo la

chiave pubblica sarà invece salvata all'interno del database. La pagina html ricevute le chiavi si presenterà così:

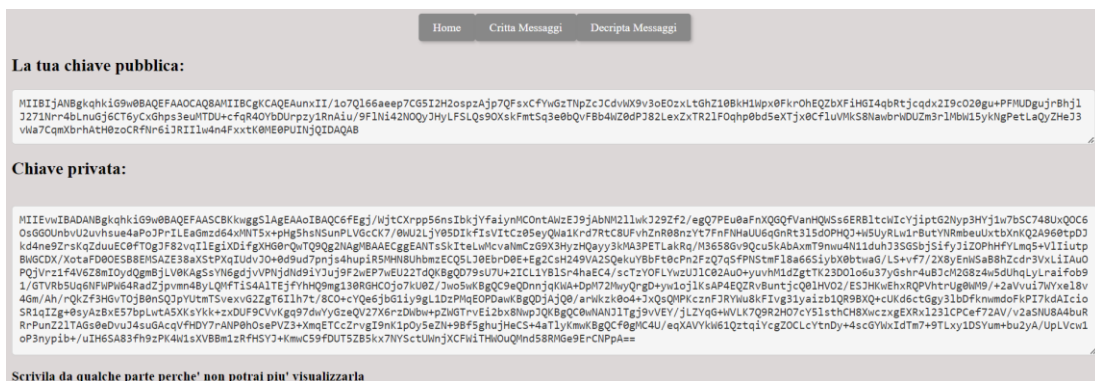


Figura 60 - Area personale dopo la generazione delle chiavi

Da adesso in poi accedendo nuovamente all'area personale questa apparirà così:



Figura 61 - Area personale

3.3.4 Critta messaggi

La pagina che permette di inserire messaggi crittati nella bacheca presenta nella parte alta tutti gli utenti che hanno generato una chiave pubblica. Attraverso le checkbox si possono scegliere quelli che potranno leggere il messaggio che verrà inserito nella text box sottostante.



Figura 62 - Pagina critta messaggi

Per il funzionamento della pagina vengono utilizzate due funzioni Lambda. La prima si occupa di creare l'elenco degli utenti disponibili. Lo fa eseguendo una scan nella

tabella chiavi e salvando ogni item nel bean Risultato che contiene una lista di liste di stringhe.

```
public Risultato handleRequest(Void map, Context context) {
    // Configura il client di Amazon Cognito
    DynamoDbClient dynamoDb = DynamoDbClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

    // Specifica i parametri per la query
    ScanRequest queryRequest = ScanRequest.builder()
        .tableName("chiavi") // Nome della tabella "ordini"
        .build();

    // Esegui la query
    ScanResponse response = dynamoDb.scan(queryRequest);
    dynamoDb.close();
    // Elabora i risultati
    Risultato ris = new Risultato();
    ris.setRisult("Fail");
    List<String> utenti = new ArrayList<String>();
    for (Map<String, AttributeValue> item : response.items()) {
        ris.setRisult("Ok");
        utenti.add(item.get("username").s());
    }
    ris.setUtenti(utenti);
    return ris;
}
```

Figura 63 - Codice ricerca utenti con chiavi pubbliche

Ricevuta la risposta nella pagina html il codice JS creerà le rispettive checkbox:

```
if(risposta.risult==="Ok"){
    messaggio.innerHTML = "";
    const container = document.getElementById("utenti");
    risposta.utenti.forEach((item, index) => {
        const checkboxLabel = document.createElement("label");
        checkboxLabel.className = "checkbox-label";
        const checkbox = document.createElement("input");
        checkbox.type = "checkbox";
        checkbox.id = `checkbox-${index}`;
        checkbox.name = "checkboxes";
        const label = document.createElement("span");
        label.textContent = item;
        checkboxLabel.appendChild(checkbox);
        checkboxLabel.appendChild(label);
        container.appendChild(checkboxLabel);
    });
} else {
    messaggio.innerHTML = "Al momento non ci sono utenti disponibili";
}
```

Figura 64 - Estratto codice JS che crea checkbox

La funzione che invece esegue la crittazione del messaggio per prima cosa recupera i dati dalla richiesta post; dopo di che per ogni destinatario richiama la funzione Lambda di ricerca della chiave pubblica mostrata in precedenza in modo da ottenere quella del destinatario con cui crittare il

```
public class ChiavePubblica {

    private String result;
    private String chiave;
    public String getResult() {
        return result;
    }
    public void setResult(String result) {
        this.result = result;
    }
    public String getChiave() {
        return chiave;
    }
    public void setChiave(String chiave) {
        this.chiave = chiave;
    }
}
```

Figura 65 - Bean ChiavePubblica

messaggio. Per chiamare una seconda funzione Lambda è necessario specificare i dati in ingresso di questa seconda funzione (payloadString) e il nome della funzione che si vuole chiamare. Dopo l'esecuzione verrà restituita una stringa che tramite la libreria gson¹⁵ può essere tradotta in un bean che in questo caso è chiamato ChiavePubblica. Una funzione Lambda per chiamarne un'altra nelle proprie policy deve avere impostato come allow l'azione: lambda:InvokeFunction.

```
public Risultato handleRequest(Map<String,String> map, Context context) {

    Risultato result = new Risultato();
    String res = "";
    Gson gson = new Gson();
    String username = (String) map.get("username");
    String messaggio = (String) map.get("messaggio");
    System.out.println("Messaggio: "+messaggio);
    String[] destinatari = ((String) map.get("destinatari")).split("@");
    DynamoDbClient ddb = DynamoDbClient.builder().build();

    for(String dest : destinatari) {
        //recupero chiave pubblica
        LambdaClient lambdaClient = LambdaClient.create();
        String payloadString = "{\"username\": \""+dest+"\"}";
        // Configura la richiesta di invocazione
        InvokeRequest invokeRequest = InvokeRequest.builder()
            .functionName("CercaChiaveGenerazione")
            .payload(SdkBytes.fromUtf8String(payloadString)) // Payload di input per la seconda funzione Lambda
            .build();
        // Invoca la seconda funzione Lambda
        InvokeResponse invokeResponse = lambdaClient.invoke(invokeRequest);
        // Estrai la risposta
        String responsePayload = invokeResponse.payload().asUtf8String();
        ChiavePubblica pub = gson.fromJson(responsePayload, ChiavePubblica.class);
    }
}
```

Figura 66 - Codice crittazione messaggio p1

Il codice prosegue con la decodifica della chiave pubblica estratta dal database che verrà utilizzata per crittare il messaggio. Quest'ultimo verrà encodato in base64 e salvato in DynamoDB nella tabella messaggi. Questa tabella è stata creata precedentemente attraverso la console di AWS e ha come chiave di partizione il destinatario del messaggio mentre come chiave di ordinamento un id. Questo id è il timestamp convertito in millisecondi come si può vedere anche dal codice. Nella tabella sono presenti anche altri attributi, cioè il mittente e letto che assume il valore no alla creazione dell'item.

¹⁵ <https://github.com/google/gson>

```

byte[] publicKeyBytes = Base64.getMimeDecoder().decode(pub.getChiave());
// Crea una specifica chiave pubblica
X509EncodedKeySpec keySpec = new X509EncodedKeySpec(publicKeyBytes);
KeyFactory keyFactory;
try {
    keyFactory = KeyFactory.getInstance("RSA");
    PublicKey publicKey = keyFactory.generatePublic(keySpec);
    Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
    cipher.init(Cipher.ENCRYPT_MODE, publicKey);
    byte[] encryptedBytes = cipher.doFinal(messaggio.getBytes());
    String crittato = Base64.getEncoder().encodeToString(encryptedBytes).replaceAll("\\r\\n", "");
    System.out.println("Crittato::: "+crittato);
    Map<String, AttributeValue> valori = new HashMap<>();
    valori.put("destinatario", AttributeValue.builder().s(dest).build());
    valori.put("id", AttributeValue.builder().s(System.currentTimeMillis()+"").build());
    valori.put("mittente", AttributeValue.builder().s(username).build());
    valori.put("messaggio", AttributeValue.builder().s(crittato).build());
    valori.put("letto", AttributeValue.builder().s("no").build());
    PutItemRequest request = PutItemRequest.builder()
        .tableName("messaggi")
        .item(valori)
        .build();

    ddb.putItem(request);
    res = res + " Messaggio aggiunto alla bacheca per: "+dest;
} catch (Exception e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
    res = res + " Fallimento messaggio per: "+dest;
    result.setResult(res);
    return result;
}
}
result.setResult(res);
return result;
}

```

Figura 67 - Codice crittazione messaggio p2

3.3.5 Decritta messaggi

Anche la pagina per decrittare i messaggi fa uso di due funzioni Lambda. La prima si occupa di eseguire la ricerca nella tabella messaggi e di restituire solo quelli indirizzati all'utente autenticato. Il codice esegue quindi una semplice query. L'oggetto Risultato è un bean che contiene una lista di liste di stringhe.

```

public Risultato handleRequest(Map<String,String> map, Context context) {
    String username = (String) map.get("username");
    // Configura il client di Amazon Cognito
    DynamoDbClient dynamoDb = DynamoDbClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

    // Specifica i parametri per la query
    QueryRequest queryRequest = QueryRequest.builder()
        .tableName("messaggi") // Nome della tabella "ordini"
        .keyConditionExpression("destinatario = :val") // Specifica la chiave di partizione
        .expressionAttributeValues(Collections.singletonMap(":val", AttributeValue.builder().s(username).build()))
        .build();

    // Esegui la query
    QueryResponse response = dynamoDb.query(queryRequest);
    dynamoDb.close();
    // Elabora i risultati
    Risultato ris = new Risultato();
    ris.setResult("Fail");
    List<List<String>> totale = new ArrayList<List<String>>();

    for (Map<String, AttributeValue> item : response.items()) {
        List<String> tupla = new ArrayList<String>();
        ris.setResult("Ok");
        tupla.add(item.get("id").s());
        tupla.add(item.get("mittente").s());
        tupla.add(item.get("letto").s());
        totale.add(tupla);
    }
    ris.setMessaggi(totale);
    return ris;
}

```

Figura 68 - Codice ricerca messaggi per specifico utente

Questa lista verrà utilizzata da JS per popolare la tabella mostrata all'interno della pagina:

```
if(risposta.result=="Ok"){
    container.innerHTML = "";
    risposta.messaggi.forEach(function (dato) {
        var row = tableBody.insertRow();
        var cell1 = row.insertCell(0);
        var cell2 = row.insertCell(1);
        var cell3 = row.insertCell(2);
        var cell4 = row.insertCell(3);

        cell1.innerHTML = dato[0];
        cell2.innerHTML = dato[1];
        cell3.innerHTML = dato[2];

        var radioButton = document.createElement("input");
        radioButton.type = "radio";
        radioButton.name = "seleziona";
        radioButton.id = dato[0];

        cell4.appendChild(radioButton);
    });
}else{
    messaggio.innerHTML = "<h3>Al momento non ci sono messaggi disponibili</h3>";
}
```

Figura 69 - Estratto codice JS che popola tabella

La seconda funzione Lambda riceve invece dalla richiesta post la chiave privata inserita dall'utente nella text box e le informazioni per recuperare il messaggio all'interno del database, cioè il destinatario e l'id del messaggio. Esegue quindi una query a DynamoDB e una volta ottenuto il messaggio lo decrittta e lo inserisce nel bean Risultato. Prima di mandare il risultato all'utente però aggiorna l'item della tabella cambiando il valore dell'attributo letto da no a sì.

```

public Risultato handleRequest(Map<String,String> map, Context context) {

    Risultato result = new Risultato();
    String username = (String) map.get("destinatario");
    String id = (String) map.get("id");
    String privata = (String) map.get("chiave").replaceAll("\\r\\n", "");
    DynamoDbClient ddb = DynamoDbClient.builder().build();

    GetItemRequest request = GetItemRequest.builder()
        .tableName("messaggi")
        .key(
            new HashMap<String, AttributeValue>() {{
                put("destinatario", AttributeValue.builder().s(username).build());
                put("id", AttributeValue.builder().s(id).build());
            }}
        )
        .build();

    // Esegui la richiesta
    GetItemResponse response = ddb.getItem(request);
    Map<String, AttributeValue> item = response.item();

    byte[] privateKeyBytes = Base64.getMimeDecoder().decode(privata);

    try {
        KeyFactory keyFactory = KeyFactory.getInstance("RSA");
        PKCS8EncodedKeySpec keySpec = new PKCS8EncodedKeySpec(privateKeyBytes);
        PrivateKey privateKey = keyFactory.generatePrivate(keySpec);
        Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding");
        cipher.init(Cipher.DECRYPT_MODE, privateKey);
        String mex = item.get("messaggio").s().replaceAll("\\r\\n", "");
        byte[] ciphertext = Base64.getMimeDecoder().decode(mex);
        byte[] decryptedMessageBytes = cipher.doFinal(ciphertext);
        String decryptedMessage = new String(decryptedMessageBytes, "UTF-8");
        result.setResult("Il messaggio e': " + decryptedMessage);

        Map<String, AttributeValue> primaryKey = new HashMap<>();
        primaryKey.put("destinatario", AttributeValue.builder().s(username).build());
        primaryKey.put("id", AttributeValue.builder().s(id).build());

        UpdateItemRequest updateRequest = UpdateItemRequest.builder()
            .tableName("messaggi")
            .key(primaryKey)
            .updateExpression("SET #attrName = :attrValue")
            .expressionAttributeNames(Collections.singletonMap("#attrName", "letto"))
            .expressionAttributeValues(Collections.singletonMap(":attrValue", AttributeValue.builder().s("si").build()))
            .build();

        ddb.updateItem(updateRequest);

    } catch (Exception e) {
        // TODO Auto-generated catch block
        result.setResult("Errore decrypt: " + e.toString());
        e.printStackTrace();
    }

    return result;
}

```

Figura 70 - Codice decrittazione messaggio

La pagina html decrittta messaggi, dopo la richiesta di decrittazione, appare così:

ID	Mittente	Letto	Seleziona
1697379191486	chris	si	☒
1697443913413	Giulio	no	☐

Aggiorna lista

Inserire chiave privata

MIIEvgIBADANBgkqhkiG9w0BAQFAASCBKggggSkAgEAAoIBAQCOTpQcELwCRga9wj1Ez9SUutxqITq5Py+7QQIKV58uA+dY5rF2uF1KwZFrgeVgPap+xx11kikeGK8tLocAN7tHb8riZNYIURmp11IPhm+A2pEF4cEe5NCnYjotgaaoQVNi3e1jRi+1fRiUc1I0eIVQ6P3yftvN51w3j/QxhVemfagc8/8508PvjdUcAHh78C2tHeaAUfUEQ6HQ8BvKV6uSOUYnpXav9p11VFJR886KRC/tYU08MU1TC47Q06jZLoF6SEtFLCgv2P2hPh11y1Sxp2AcuNvQkqDZ01s80HusYKgyrIfVoc116im6GagXwi1kVyrfa5R7jAgBAAEAgg8BAg7IR1be2BUCVnaih/qcrMv12hwiRwrra7F1UpH8QpA27KXfzjb46jLDK6+TetXSHBoeRxyGvAs57yaqW158t36WmWq8EkWpXNVztUPV3iuOv==S1IdQF8naphfARzxcFfARrL70z+UhuCUnQJlacjXm2Fxe2uUuW5RebFus4EUGcc0NbFz8MHR+TNbmb88CKPDXDK2Pu76gKu1jXN3c6QD3rBA88PAnef1yttPt3buM9D1KG4a5J1AURwTXnBe3K1xhkn/U9s5GV5x9InF43UZykqCfqaDmBvQponSTQ1nhUI/+NGUH4ptkHuo1sUd1Q1afY1xq7tcLFJAECgYE4ARsfyrKuv3pQ6+2vz3x31WYOTe80NmZ/SP1jUHM3F5tvyU11RwRBF/K08t6uXAed821o/h7/VT6Pj/uYpe41i8t5aL21EDg3c1YV1EqA8sEKwv3diNpsdQP6fBwLcp0W8k3xL3w20n1f013685bxiCsn0/19rc+ztN91ECgYEAv2Y0xAZx/6wIac1JnT8mk0yWULjpu2HueUfzkuuU13ap8YOSIzfGw+s9H3f1XkpZZFY0Qk3Hpi1WYTFdnz2BC4eNPd+pe/FobE7DRFZ0pIw3wft1bly1IvM5zCOObuSmXDP6PKnbUsgr21U699c07Bh+BIDKXWAO8MCGy8eeVT7b69TnVOLT+CuCu15TR4tGmkaVAIVYR7DUegjar1ECN82H0Zk086P44vz4dp/muDOj1HZA+2anxw8n4svGFAG1JcqN2RokU72Du75fyKwU1h75e3ZQvNBGTesAq9R6E8OCp7zb22C54syomaC1KxR18c20vygE1EQBgKc213jMzr1LoLa3fjd3xxxyAL/SOOS5Wu3VrtVPvPdu/56HgADPJP5DV8B44BENKLC7B2zmiBe5GPEQQTa9+TdzgOA/8Xv2cN9ADHdCYPjvkg12nL84fg1n6011AeR64dyIv+bpko84K/IxbyIzvmAQp+mQpoteUHZ9jMQK8gfj31iPVAcxQYahUXVHzm1vKKWj33/HL8ocFgnlQ7z3HT4dr2X7T2zCm21e411wsg8EG6Pxy7qAphi6QZVehY3FTU4xothr/evW4gASg+ooEykuBzahr21woyk10fAepfELK8WoZ1G0vX1AKBqmrvVTCUTCUQvbiop6k

Decritta

Il messaggio e': Ciao tutto bene?

Il pulsante aggiorna lista non fa altro che richiamare la prima funzione Lambda descritta in questo capitolo.

3.3.6 Home

Nella pagina home vengono visualizzate le due tabelle contenenti una le chiavi pubbliche generate degli utenti e l'altra i messaggi inseriti nella bacheca. Anche in questo caso si utilizzano due funzioni Lambda che eseguono una scan nella tabella di DynamoDB.

Questo è il codice che restituisce le chiavi pubbliche presenti nel sistema:

```
public Risultato handleRequest(Void map, Context context) {
    // Configura il client di Amazon Cognito
    DynamoDbClient dynamoDb = DynamoDbClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

    // Specifica i parametri per la query
    ScanRequest queryRequest = ScanRequest.builder()
        .tableName("chiavi") // Nome della tabella "ordini"
        .build();

    // Esegui la query
    ScanResponse response = dynamoDb.scan(queryRequest);
    dynamoDb.close();
    // Elabora i risultati
    Risultato ris = new Risultato();
    ris.setRisult("Fail");
    List<List<String>> totale = new ArrayList<List<String>>();
    for (Map<String, AttributeValue> item : response.items()) {
        List<String> tupla = new ArrayList<String>();
        ris.setRisult("Ok");
        tupla.add(item.get("username").s());
        tupla.add(item.get("chiave").s());
        totale.add(tupla);
    }
    ris.setChiavi(totale);
    return ris;
}
```

Figura 71 - Codice ricerca chiavi pubbliche

Questo è invece il codice che mostra i messaggi che gli utenti hanno aggiunto alla bacheca:

```
public Risultato handleRequest(Void map, Context context) {
    // Configura il client di Amazon Cognito
    DynamoDbClient dynamoDb = DynamoDbClient.builder()
        .region(Region.EU_NORTH_1)
        .build();

    // Specifica i parametri per la query
    ScanRequest queryRequest = ScanRequest.builder()
        .tableName("messaggi") // Nome della tabella "ordini"
        .build();

    // Esegui la query
    ScanResponse response = dynamoDb.scan(queryRequest);
    dynamoDb.close();
    // Elabora i risultati
    Risultato ris = new Risultato();
    ris.setResult("Fail");
    List<List<String>> totale = new ArrayList<List<String>>();
    for (Map<String, AttributeValue> item : response.items()) {
        List<String> tupla = new ArrayList<String>();
        ris.setResult("Ok");
        tupla.add(item.get("destinatario").s());
        tupla.add(item.get("id").s());
        tupla.add(item.get("letto").s());
        tupla.add(item.get("messaggio").s());
        tupla.add(item.get("mittente").s());
        totale.add(tupla);
    }
    ris.setMessaggi(totale);
    return ris;
}
```

Figura 72 - Codice ricerca messaggi del sistema

La pagina home appare quindi in questo modo:

Login Registrati				
Username		Chiave		
chris		MIIBJjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAnuxDIIo7Q66aesp7CG5I2H2oispzAjp7QFnxCTYwGzTSpZcJcdvWX9v3oEOxLlGhZ10BkH1Wpx0Fkt		
Giulio		MIIBJjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAlivav7kR81xeijprLyMjEcogAUFGbPj4YGuK4KTTPsJ6zy0BSN2ZIDFkRqkj2UvWv9g/zgNOYjqEppelH		
Alessandro		MIIBJjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAbf8JESYjllVPYvM6uJW3Gf6qb6Q6+fmIgfZlUmwXLfvknT4CmZqDlbuhz31TA7b6066mawJEA2XvW		
Maria		MIIBJjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAnlida4XDYWFQE0hlovV3pEIKSQMZ3px+ynaqF1i2MsnL6vTPYlth3F3sY99ecp6Wyd0IWFRCPCoo6		
Mittente	Destinatario	id	letto	messaggio
chris	chris	1697379191486	si	KMb98bAwnIedF9+cfZIVNlXV5mlbP6CILeCQ0f0D4C1EPeXQterWN0MuXVn4RzJgBXex2f6nJPYGy/ooY3HERB2TeK82HK2votegKCh/XnLQD1eu2D1rbukFQj
Giulio	chris	1697443913413	no	XF0MQWKKfMAgrfBf3cM294W3sqGkXAPRQAzOE55f6H9QFD6Y7czbyf4+wKCBQMPTW22IfaR457CmqvV6Ufj1AKAT4/haq9E4qm/O2A25ALercZ4L7BSKLYP
chris	Alessandro	1697441808848	no	bebPyK64JEH0R7R9DGsAnlMG+0LzXUTaE/ZHo5m7eUJHMUSa3xeL0PNCzaReUwrZyRp1A5lkfpxUDrFN5WjSseDOFo2oRNx6dH1eq9XgdglFby3ssI8CoxDqIotXj
chris	Alessandro	1697441814746	no	cHzeXmL0J5NH/bafpXeedIs1U7knE419EP2HhSalFezqc635bFCugbejMVAD5ZIsPkNlUMX4dCOXW8X2C/7WlkAgsKQsQdKj/B1XaeH3Rp9fwY8MGwR/gaw9+D
Giulio	Alessandro	1697443927461	no	ZmUwtjXJAoaSmmNm2TP3PmZelZKwsg2Ue8dfwT17IitL3uz2m3VbheclJVKOSiKempT0Jw4IwGhhVZNYtbh9jipTb3Fwm0tyzFhXsyVV0dSSD8HbPeAmbse+4Woc
Alessandro	Maria	1697443942261	no	a74uNHlGo8HxGUoM+IQA+1ILrUBzrPke7qjFKu/85z04WP+vLxjea+Y9EaH6UW0FHokgGVzT62DWLTrbcinJ8QDpBlpzkW+xwVHx5R2oH4VWbbSaxXueNj4InXS

Figura 73 - Pagina home

3.3.7 Caricamento delle pagine

Tutte le pagine html mostrate precedentemente sono state inserite in un unico file zip contenete anche un foglio di stile css per dare una grafica migliore all'applicazione. Il file è poi stato caricato su AWS Amplify. La pagina home è chiamata index in modo che quando ci si collega al dominio su cui è hostata l'applicazione quella sarà la prima pagina che verrà visualizzata. La pagina redirect.html contiene un codice JS che indirizza l'utente alla home. Quella pagina viene richiamata da una API Gateway che viene a sua volta chiamata dalla pagina login.html quando quest'ultimo ha successo. Questo passaggio è stato inserito per avere magari la possibilità di aggiungere ulteriori controlli in una fase successiva all'autenticazione. Magari in base a specifici parametri la pagina redirect.html potrebbe non indirizzare l'utente alla home ma ad un altro servizio.

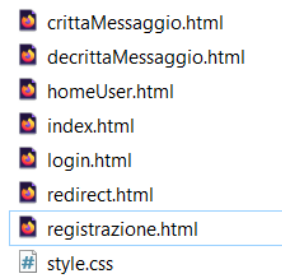


Figura 74 - Pagine dell'applicazione

3.4 Analisi delle performance

Una delle maggiori problematiche riscontrate durante l'utilizzo delle varie funzioni Lambda è il tempo necessario per l'inizializzazione delle stesse quando ci si trova di fronte ad un avvio a freddo ma anche il tempo necessario per leggere i dati scritti sul DB se questi non vengono usati per diverso tempo. Problematica descritta anche nei capitoli iniziali. Questa latenza non è presente se la funzione viene chiamata spesso perché resta attiva ma nel caso in cui per un ampio lasso di tempo non ci sia nessuna chiamata questa si disattiva. L'utente deve quindi aspettare più tempo per ricevere il risultato e questo può generare fastidio. Non si parla certo di attese lunghe minuti, il lasso di tempo va da pochi secondi fino a circa un minuto, ma queste attese posso essere comunque frustranti.

Nell'applicazione presentata al capitolo 3 le richieste alle varie funzioni Lambda vengono effettuate tramite chiamate AJAX questo vuol dire che la pagina rimane attiva anche quando la risposta non è ancora arrivata. Per evitare di lasciare l'utente

disorientato viene sempre mostrata a schermo una stringa che indica all'utilizzatore che qualcosa sta calcolando o sta elaborando in modo anche da evitare che quest'ultimo clicchi varie volte un bottone pensando che nulla stia succedendo.

Figura 75 - Stringa caricamento

Per analizzare al meglio il tempo di esecuzione delle

Run 1	24,881 s
Run 2	1,119 s
Run 3	0,937 s
Run 4	0,611 s
Run 5	0,536 s
Run 6	0,392 s
Run 7	0,387 s
Run 8	0,547 s
Run 9	0,353 s
Run 10	0,342 s

Tabella 1 - Test crittazione messaggio

funzioni Lambda si possono osservare le tracce generate da AWS X-Ray. Come primo esempio

```
{
  "username": "value1",
  "messaggio": "value2",
  "destinatari": "chris"
}
```

Figura 76 - Test 1

verrà analizzata la funzione che si occupa di crittare i messaggi visto che si tratta di una delle operazioni più costose del sistema anche perché al suo interno utilizza a sua volta un'altra funzione Lambda. I risultati di questo e dei seguenti test saranno effettuati su dieci richieste consecutive. La prima serie di test eseguita critta il messaggio per un solo destinatario. Dai dati raccolti si può notare che la media di esecuzione delle operazioni si attesta sui 3,01 secondi. Per quanto riguarda la

varianza questa si attesta su 59,12 per cui i tempi di esecuzione si distaccano abbastanza dalla media. Quello che salta subito all'occhio è che la prima esecuzione è quella più lenta visto che è presente anche una fase di inizializzazione. Già dalla seconda i tempi di attesa calano di molto.

▼ crittaStringa	AWS::Lambda::Function			
crittaStringa	OK	-	24.88s	
Initialization	OK	-	2.99s	
Invocation	OK	-	24.88s	
Overhead	OK	-	0ms	

Figura 77 - Tempo di esecuzione Run 1 crittazione messaggio con singolo destinatario

Si prova ora a vedere come cambiano le prestazioni

Run 1	27,419 s
Run 2	1,766 s
Run 3	1,077 s
Run 4	1,178 s
Run 5	1,162 s
Run 6	1,055 s
Run 7	1,327 s
Run 8	0,911 s
Run 9	0,941 s
Run 10	0,969 s

Tabella 2 - Test crittazione messaggio su più destinatari

nel caso in cui i destinatari aumentino, in questo test se

```
{
  "username": "value1",
  "messaggio": "value2",
  "destinatari": "chris@Giulio@Maria"
}
```

Figura 78 - Test 2

ne selezionano tre. Il messaggio sarò quindi leggibile da più utenti e questo comporterà un numero più alto di crittazioni ma anche di chiamate alla funzione Lambda che si occupa di trovare le chiavi pubbliche. In questo caso il tempo medio per l'esecuzione è di 3,78 secondi. È quindi presente un aumento rispetto all'esecuzione su un unico destinatario. Per quanto riguarda la varianza questa si attesta su 69,04. I tempi di esecuzione in questo caso si disperdono maggiormente rispetto al test precedente.

▼ crittaStringa AWS::Lambda::Function				
crittaStringa	OK	-	27.42s	
Initialization	OK	-	3.25s	
Invocation	OK	-	27.42s	
Overhead	OK	-	0ms	

Figura 79 - Tempo di esecuzione Run 1 crittazione messaggio con più destinatari

Run 1	11,827 s
Run 2	0,388 s
Run 3	0,671 s
Run 4	0,772 s
Run 5	0,501 s
Run 6	0,173 s
Run 7	0,231 s
Run 8	0,423 s
Run 9	0,215 s
Run 10	0,158 s

Tabella 3 - Test elenco messaggi della bacheca

Si prova ora ad analizzare una delle funzioni più semplici presenti all'interno dell'applicazione, cioè la funzione che restituisce tutti i messaggi inseriti nella bacheca. Questa operazione esegue solo una scan all'interno di una singola tabella di DynamoDB. I tempi di esecuzione rispetto alla funzione precedente sono ovviamente più bassi ma si può notare come la prima richiesta, cioè quella che avviene a freddo, rimane molto più lenta rispetto a quelle successiva. Il tempo di esecuzione medio è di 1,54 secondi mentre la varianza è di 13,12. Anche la dispersione dei tempi rispetto al valor medio si è ridotta.

Infine, si analizza una funzione Lambda che non interagisce con il database ma con il

Run 1	14,298 s
Run 2	1,244 s
Run 3	0,606 s
Run 4	0,575 s
Run 5	0,449 s
Run 6	0,414 s
Run 7	0,411 s
Run 8	0,489 s
Run 9	0,411 s
Run 10	0,438 s

servizio Amazon Cognito. La funzione in questione è quella che esegue l'autenticazione degli utenti. In questo caso il tempo di esecuzione medio è di 1,93 secondi mentre la varianza è di 18,94. Anche in questo caso i valori del tempo di esecuzione si distaccano meno dal valor medio rispetto alla funzione in cui vengono crittati i messaggi. Il sistema AWS si comporta quindi molto bene nel caso in cui ci sia una forte attività del sistema. Se però sono presenti dei momenti di idle la responsività della propria applicazione è destinata a calare.

Tabella 4 - Test login utente

Questo perché le funzioni Lambda vengono disattivate e alla chiamata successiva necessitano di un periodo di attivazione dell'ambiente in cui dovranno poi eseguire.

Conclusioni

Lo scopo principale di questo elaborato era esplorare i servizi offerti da Amazon Web Services per poter sviluppare un'applicazione web serverless. In questo progetto di tesi ho toccato con mano gli aspetti positivi e negativi di lavorare con il cloud computing, un ambiente con il quale non avevo nessuna familiarità. Sicuramente la grande quantità di documentazione offerta da Amazon mi ha aiutato ad apprendere abbastanza in fretta il funzionamento dei vari servizi e i meccanismi necessari per poter implementare del codice funzionante. Anche le interfacce grafiche delle console AWS sono abbastanza user friendly è questo è sicuramente un aspetto positivo. Anche se alcune volte la quantità di sezioni e impostazioni è così elevata, visto la forte personalizzazione dei servizi, da richiedere del tempo prima di riuscire a trovare velocemente ciò che si sta cercando. Un elemento sicuramente vantaggioso è la facilità con cui è possibile interfacciare i vari prodotti offerti dalla nuvola. Con un paio di righe di codice è possibile far interagire altri servizi e scambiarsi informazioni senza doversi occupare di gestire credenziali o connessioni. Creare quindi un'infrastruttura in AWS è abbastanza semplice anche per chi non ha mai usato questi strumenti. Uno degli aspetti più critici però è stato il debug dell'applicazione. Nel caso in cui ci fossero dei problemi di esecuzione non era possibile eseguire il codice passo passo all'interno della console AWS e questo spesso ha richiesto di inserire varie stampe nel log per sapere effettivamente cosa stesse succedendo. In un caso ho anche trovato utile costruire un main in locale per capire cosa non andasse nella funzione Lambda che avrei dovuto caricare sul cloud. Per quanto riguarda invece l'analisi delle performance, dai vari test è emerso che le applicazioni sono davvero molto reattive nel caso in cui ci sia un flusso di traffico elevato. In caso contrario il tempo di attesa per le varie esecuzioni aumenta richiedendo anche diversi secondi per completare l'esecuzione. Questo è un aspetto negativo del non acquistare uno spazio di esecuzione che è sempre disponibile ad accettare richieste. Spesso può essere fastidioso anche in fase di test visto che nel caso in cui si ricarica un codice all'interno di una funzione Lambda, anche molto semplice, è necessario attendere anche dieci secondi prima di ottenere il risultato a causa dell'inizializzazione.

Il lavoro di tesi permette ovviamente diversi sviluppi futuri. I servizi AWS analizzati sono solo una parte di quelli che si possono utilizzare per la costruzione di applicazioni serverless. Si potrebbe quindi aggiungere un approfondimento sul funzionamento di altri prodotti come ad esempio Amazon Cloudfront. Anche l'aspetto della sicurezza delle applicazioni non è stato trattato in questo lavoro. Ci sono ovviamente servizi costruiti ad hoc per la gestione di reti interne, di firewall e di DNS. Anche il confronto con altri provider come, ad esempio, quello di Google potrebbe essere un aspetto interessante da analizzare. Si potrebbe mostrare un paragone fra le performance della stessa applicazione implementata sulle due infrastrutture cloud. Anche l'app presentata al capitolo 3 potrebbe essere espansa con ulteriori funzionalità. Ad esempio, la possibilità di inserire messaggi visibili a tutti che però vengono firmati con la chiave privata del mittente in modo che i vari destinatari possano essere certi che quel testo sia stato davvero inserito da quello specifico utente.

Indice delle figure

Figura 1 - Mercato Cloud.....	7
Figura 2 - Servizi AWS	11
Figura 3 - Logo AWS Lambda	11
Figura 4 - Creazione di funzione Lambda	12
Figura 5 - Esempio codice funzione Lambda	13
Figura 6 - Struttura del codice	13
Figura 7 - Impostazioni di runtime	14
Figura 8 - Parametri evento	14
Figura 9 - Risultato di un test di una funzione Lambda.....	15
Figura 10 - Monitoraggio X-Ray	15
Figura 11 - Variabili d'ambiente	16
Figura 12 - Logo AWS Identity and Access Management (IAM).....	16
Figura 13 - Set autorizzazioni amministratore.....	17
Figura 14 - Creazione di un'organizzazione	17
Figura 15 - Struttura organizzazione	18
Figura 16 - Policy	18
Figura 17 - Policy in formato JSON	18
Figura 18 - Interfaccia per creare policy.....	19
Figura 19 - Sistema di analisi degli accessi	20
Figura 20 - Logo Amazon API Gateway	20
Figura 21 - Architettura Amazon API Gateway	20
Figura 22 - Deploy API	21
Figura 23 - Test API	21
Figura 24 - Richiesta POST	22
Figura 25 - Provider di autenticazione.....	22
Figura 26 - Definizione dell'autorizzazione dell'API.....	23
Figura 27 - Accesso negato alla funzione Lambda.....	23
Figura 28 - Richiesta con token	23
Figura 29 - Logo AWS Amplify.....	24
Figura 30 - Installare CLI	24
Figura 31 - Chiavi di accesso IAM.....	25
Figura 32 - Creazione nuovo progetto Amplify.....	25
Figura 33 - Hosting environments	25
Figura 34 - Esempio pagine html.....	26
Figura 35 - Richiesta AJAX.....	26
Figura 36 - Applicazione Amplify.....	26
Figura 37 - Logo Amazon Cognito.....	27
Figura 38 - Interfaccia login	27
Figura 39 - Definire come Cognito distribuisce il token	28
Figura 40 - Logo Amazon DynamoDB	29

Figura 41 - Tabella DynamoDB	29
Figura 42 - Rappresentazione in JSON di un item	30
Figura 43 - Query in DynamoDB	30
Figura 44 - Codice Java che aggiunge item ad una tabella.....	31
Figura 45 - Eliminare item da una tabella di DynamoDB	32
Figura 46 - Policy per DynamoDB	32
Figura 47 - Logo AWS X-Ray.....	33
Figura 48 - Traccia AWS X-Ray	33
Figura 49 - Messaggio di verifica	35
Figura 50 - Codice registrazione.....	36
Figura 51 - Policy registrazione.....	36
Figura 52 - Pagina di registrazione	37
Figura 53 - Bean risultato	37
Figura 54 - Codice Login.....	38
Figura 55 - Pagina Login	38
Figura 56 - Area personale prima di generare chiave pubblica	39
Figura 57 - Codice JS che mostra o la chiave pubblica o chiede all'utente di generarla	39
Figura 58 - Codice per la ricerca delle chiavi pubbliche	40
Figura 59 - Codice generazione chiavi asimmetriche.....	40
Figura 60 - Area personale dopo la generazione delle chiavi.....	41
Figura 61 - Area personale.....	41
Figura 62 - Pagina critta messaggi.....	41
Figura 63 - Codice ricerca utenti con chiavi pubbliche	42
Figura 64 - Estratto codice JS che crea checkbox	42
Figura 65 - Bean ChiavePubblica	43
Figura 66 - Codice crittazione messaggio p1.....	43
Figura 67 - Codice crittazione messaggio p2.....	44
Figura 68 - Codice ricerca messaggi per specifico utente	44
Figura 69 - Estratto codice JS che popola tabella.....	45
Figura 70 - Codice decrittazione messaggio	46
Figura 71 - Codice ricerca chiavi pubbliche	47
Figura 72 - Codice ricerca messaggi del sistema.....	48
Figura 73 - Pagina home	48
Figura 74 - Pagine dell'applicazione	49
Figura 75 - Stringa caricamento.....	50
Figura 76 - Test 1.....	50
Figura 77 - Tempo di esecuzione Run 1 crittazione messaggio con singolo destinatario	50
Figura 78 - Test 2.....	51
Figura 79 - Tempo di esecuzione Run 1 crittazione messaggio con più destinatari..	51

Indice delle tabelle

Tabella 1 - Test crittazione messaggio	50
Tabella 2 - Test crittazione messaggio su più destinatari	51
Tabella 3 - Test elenco messaggi della bacheca	51
Tabella 4 - Test login utente	52

Bibliografia

[1] “Cloud storage vs. on-premises servers: 9 things to keep in mind” [Online]
Available: <https://www.microsoft.com/en-ww/microsoft-365/business-insights-ideas/resources/cloud-storage-vs-on-premises-servers>

[2] “Cos’è il cloud computing” [Online] Available: <https://aws.amazon.com/it/what-is-cloud-computing/>

[3] “Cloud Computing Market Share” [Online] Available:
<https://www.t4.ai/industry/cloud-computing-market-share>

[4] “Cloud computing con AWS” [Online] Available:
<https://aws.amazon.com/it/what-is-aws/>

[5] “Che cos’è il computing serverless?” [Online] Available:
<https://www.cloudflare.com/it-it/learning/serverless/what-is-serverless/>

[6] “Pro e contro di passare ad un'architettura serverless” [Online] Available:
<https://blog.sparkfabrik.com/it/architettura-serverless-pro-e-contro>

[7] “Why use serverless computing? | Pros and cons of serverless” [Online]
Available: <https://www.cloudflare.com/it-it/learning/serverless/why-use-serverless/>

[8] “Caratteristiche di AWS Lambda” [Online] Available:
<https://aws.amazon.com/it/lambda/features/>

[9] “Prezzi di AWS Lambda” [Online] Available:
<https://aws.amazon.com/it/lambda/pricing/>

[10] “Domande frequenti su AWS Lambda” [Online] Available:
<https://aws.amazon.com/it/lambda/faqs/>

[11] “What is IAM?” [Online] Available:
<https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>

[12] “IAM Access Analyzer ti guida verso le autorizzazioni con privilegi minimi”
[Online] Available: <https://aws.amazon.com/it/iam/features/analyze-access/>

[13] “Amazon API Gateway” [Online] Available: <https://aws.amazon.com/it/api-gateway/>

[14] “AWS Amplify” [Online] Available: <https://aws.amazon.com/it/amplify/>

[15] “Amazon Cognito” [Online] Available: <https://aws.amazon.com/it/cognito/>

[16] “What is Amazon Cognito?” [Online] Available:
<https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>

[17] “What is Amazon DynamoDB?” [Online] Available:
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>

[18] “What Is the Difference Between Partition Key and Sort Key in Amazon DynamoDB” [Online] Available: <https://saturncloud.io/blog/what-is-the-difference-between-partition-key-and-sort-key-in-amazon-dynamodb/>

[19] “What is AWS X-Ray?” [Online] Available:
<https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html>